Testing code

Paco

2025-12-05

Table of contents

1 Bagging 2

2 Random forest 6

3 Boosting 15

4 Support Vector Machines: Maximal Margin Classifier 18

5 Support Vector Machines: Support Vector Classifier 23

6 Support Vector Machines 28

7 Deep learning: Single hidden layer neural networks 35

8 Deep Learning: Deep Neural Networks 41

9 Deep learning: Fitting neural networks and back-propagation 44

10 Deep learning: Regularization for neural networks 56

11 Deep learning: Convolutional Neural Networks 59

12 Deep learning: Further topics 73

1 Bagging

1.1 Bagging: main idea

- **Bootstrap aggregation**, or **bagging**, is a general-purpose procedure for reducing the variance of a statistical learning method. It reduces variability by training multiple models on different bootstrap samples and averaging their predictions. Since each model sees a slightly different subset of the data, their individual errors tend to cancel out when aggregated, leading to a smoother and more stable estimator.
- Given a set of B independent observations $z_1, \dots, z_B$ each with variance σ^2, the variance of the mean

$$\bar{z} = \frac{1}{B} \sum_{i=1}^{B} z_i$$

  is σ^2/B.
- We can therefore reduce the variance by averaging a set of predictions. However, this is not practical because we usually do not have access to multiple training sets.
- Instead, we use the bootstrap by taking repeated samples from the training data

$$Z = \{(x_i, y_i),\ i = 1, \dots, n\}.$$

Bagging works by creating multiple bootstrap samples from the original dataset, training a separate model on each, and averaging their predictions. This aggregation stabilises the model by lowering variance without significantly increasing bias.

1.2 The bootstrap

- Let B be the number of desired bootstrap datasets $Z^{*1}, \dots, Z^{*B}$.
- For $b = 1, \dots, B$:
  - Sample n observations from Z **with replacement**.
  - This is the bth bootstrap dataset, denoted Z^{*b}.
- Each bootstrap dataset contains on average 63.2% of the original samples.

The bootstrap procedure generates multiple resampled datasets by drawing with replacement from the original data. Each sample replicates the variability inherent in the population, allowing us to estimate model variance or prediction stability without requiring new data. This approach underlies bagging and several ensemble methods.

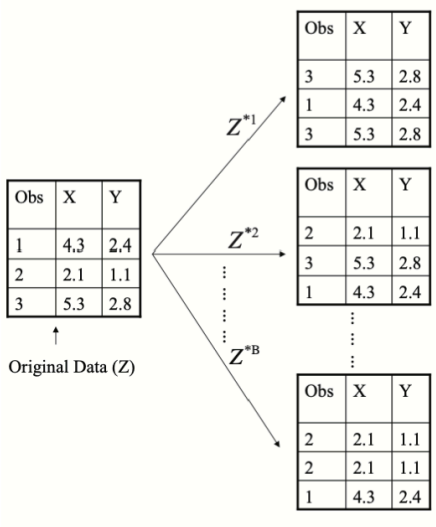

Figure 1: Bootstrap sampling

1.3 Bagging

- Let $Z^{*1}, \ldots, Z^{*B}$ be B bootstrap datasets from the training data. For all $b = 1, \ldots, B$, we can fit our predictive regression model $\hat{f}^{*b}$ on the bth bootstrap dataset Z^{*b}.
- We then average all the predictions to obtain a single bagged model

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}^{*b}(x)$$

  which is called **bagging**. In other words, we take all the B predictions made by the B models trained on different bootstrap samples. Then average those predictions to get a final one ($\hat{f}_{\text{bag}}(x)$).
- If the individual models $\hat{f}^{*b}$ have low bias but high variance, the combined estimator $\hat{f}_{\text{bag}}$ retains a similar bias but exhibits lower variance.
- In practice, we use a value of B sufficiently large for the error to stabilise. Typically, a value between 100 and 1000 is sufficient.

Bagging therefore reduces variance through model aggregation while preserving the bias of the underlying learners, resulting in more stable and robust predictions.

1.4 Out-of-bag error estimate

- There is a straightforward way to estimate the test error of a bagged model, called the **out-of-bag (OOB) error estimate**.

- The key idea of bagging is that the model is repeatedly fitted to bootstrap subsets of the data, which contain on average about two-thirds of the observations (≈ 63%).
- The remaining one-third (≈ 37%) of the observations not used to fit a given bagged model are referred to as the **out-of-bag observations**.
- We can predict the response for the ith observation using all models $\hat{f}^{*b}$ with $b \in I_{x_i} = \{b = 1, \dots, B : (x_i, y_i) \notin Z^{*b}\}$, i.e. the models for which the observation was OOB. This yields around $B/3$ predictions for the ith observation, which we average:

$$\text{Err}_{\hat{f}_{\text{bag}}}(x_i) = \frac{1}{|I_{x_i}|} \sum_{b \in I_{x_i}} (y_i - \hat{f}^{*b}(x_i))^2.$$

  In other words, this is *the mean of the squared prediction errors made by all the models* b in I_{x_i} (the models for which x_i was OOB).
- The average over all n observations gives the overall OOB error estimate:

$$\text{Err}_{\hat{f}_{\text{bag}}} = \frac{1}{n} \sum_{i=1}^{n} \text{Err}_{\hat{f}_{\text{bag}}}(x_i).$$

  On average, how well does the bagged model generalise to unseen data?
- For large B, this estimator can be shown to be equivalent to the Leave-One-Out Cross-Validation (LOOCV) estimator, but in bagging, it comes for free.

The OOB error provides an internal validation mechanism without requiring an explicit test set, making it especially efficient for ensemble methods such as Random Forests.

1.5 Bagging for trees

- Classification and regression trees are perfectly suited for bagging, since their poor predictive accuracy comes from their high variance.
- For a regression problem, [1] for all B bootstrap training sets we grow a large tree **without pruning** to obtain a high-variance predictive model. We then average all these (possibly overfitted) trees to obtain a bagged model with reduced variance.
- This has shown to give impressive improvements in prediction accuracy, though at the cost of interpretability.
- Importantly, B is not a classical tuning parameter, since a large B will not overfit the data; the error will simply stabilise.
- For classification, we fit B classification trees $\hat{G}^{*b}$ and then choose the predicted class by majority vote, that is

$$\hat{G}_{\text{bag}}(x) = \arg \max_{g=1,\dots,q} \frac{1}{B} \sum_{b=1}^{B} 1\{g = \hat{G}^{*b}(x)\}$$

[1] In this context, a regression problem refers to a supervised learning task where the target variable y is continuous. In contrast, a classification problem deals with categorical outputs.

- $\hat{G}_{\text{bag}}(x)$: the **final bagged classifier**, i.e. the class label the **ensemble of trees** assigns to x after combining all their votes.
- $\arg\max_{g=1,\ldots,q}$: we look across all possible classes and choose the one with the **largest average vote** from the trees.
- $\frac{1}{B}$: we compute the **fraction of trees** that predict each class g.
- $\hat{G}^{*b}(x)$: the **prediction of the** bth tree for observation x.
- $1\{g = \hat{G}^{*b}(x)\}$: for a given class g, this equals 1 if the bth tree predicts class g, and 0 otherwise. The sum $\sum_{b=1}^{B} 1\{g = \hat{G}^{*b}(x)\}$ therefore counts how many trees voted for g.

- Bagging is therefore especially effective for high-variance, low-bias models such as decision trees, as averaging across bootstrap replicates substantially reduces their prediction variance.

1.6 Example: bagging and variance reduction

To illustrate the effect of bagging, we return to the **heart disease** dataset, where decision trees are used to predict whether a patient has heart disease. As seen earlier, a single unpruned tree has high variance: small changes in the data can lead to very different tree structures.

Bagging mitigates this instability by repeatedly resampling the training data and averaging the resulting models.

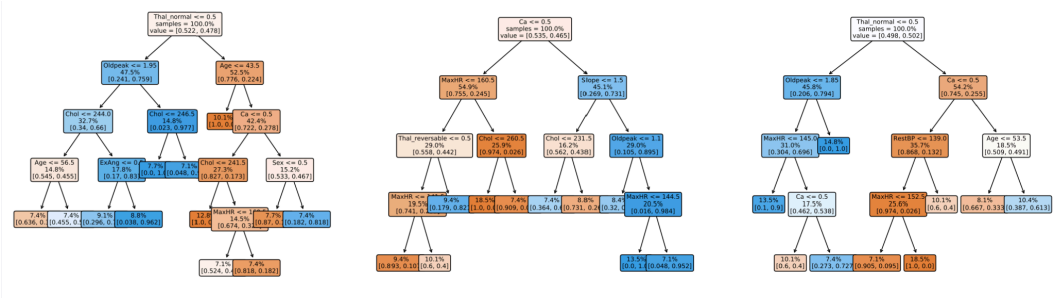

Figure 2: Bootstrapped trees

In Figure 2, three trees are trained on different bootstrap samples drawn **with replacement** from the same dataset. Each tree follows a different structure and makes slightly different predictions, showing the variability of high-variance learners such as decision trees.

By averaging the predictions from all these trees, bagging produces a single, more stable model with reduced variance. The gain from aggregation is evident in the out-of-bag (OOB) error plot below.

In Figure 3, the **OOB error rate** decreases rapidly as the number of trees increases, then stabilises once variance reduction saturates. This shows how bagging improves generalisation performance by averaging across many slightly different models rather than relying on a single, unstable one.

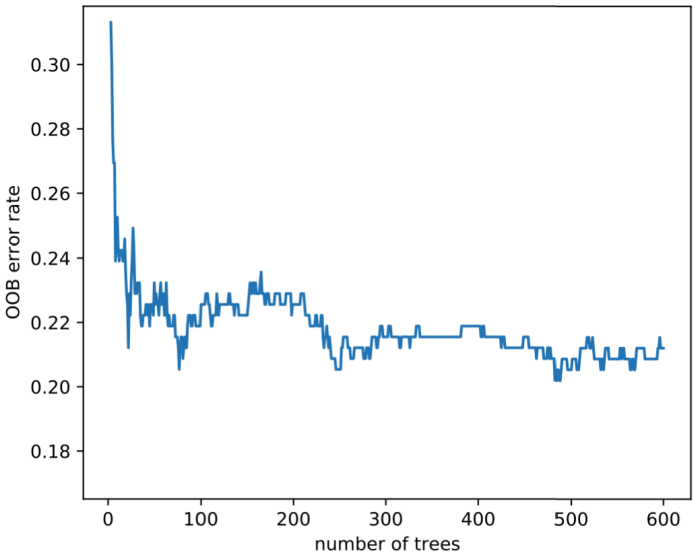

Figure 3: OOB error rate for bagged trees

2 Random forest

2.1 Variance reduction and correlation

- Recall that given a set of B **independent** observations $z_1, \ldots, z_B$, each with variance σ^2, the variance of their mean is

$$\bar{z} = \frac{1}{B}\sum_{i=1}^{B} z_i$$

  and the variance of $\bar{z}$ is σ^2/B.
- This result holds only if the z_i are independent. If they are **correlated**, the variance reduction achieved by averaging becomes much smaller.
- In bagging, each model is trained on a bootstrap sample of the data. These samples overlap substantially, so the fitted trees are often **correlated**. As a result, the averaging in bagging does not reduce variance as effectively as it would if the models were completely independent.
- To understand this, consider $z_i \sim \mathcal{N}(0, 1)$ and three levels of correlation between them:

  1. **Independence:** $\text{cor}(z_i, z_j) = 0$ for all i, j.
  2. **Weak dependence:** $\text{cor}(z_i, z_{i+1}) = 0.3$.
  3. **Strong dependence:** $\text{cor}(z_i, z_{i+1}) = 0.8$.

As seen in Figure 4, correlation among observations (or models) greatly affects how quickly variance decreases when averaging.

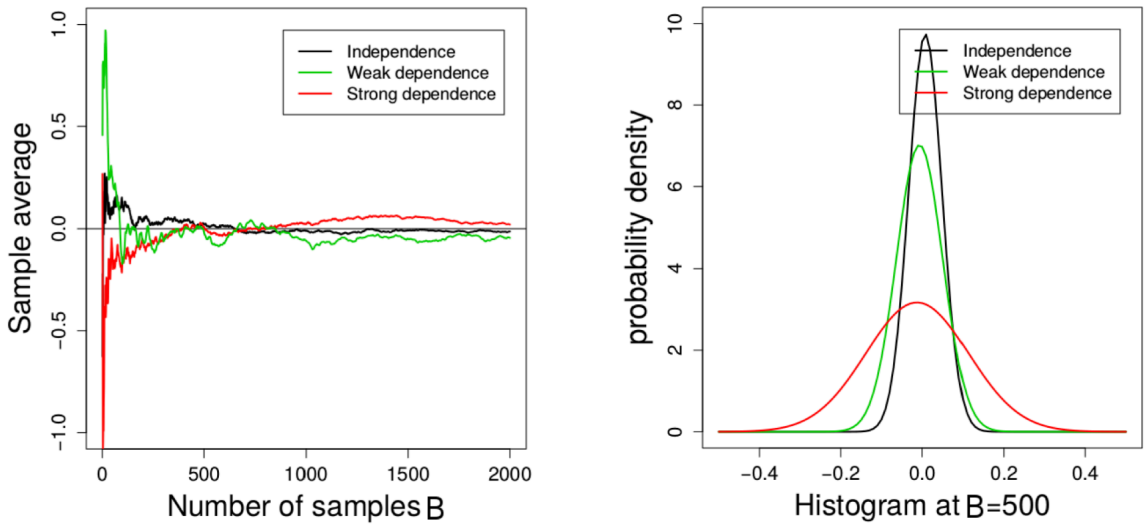

Figure 4: Effect of Correlation on Variance Reduction

The **left plot** shows the sample average as the number of samples B increases. When the observations are independent (black line), the average quickly stabilises near zero. With weak (green) or strong (red) dependence, convergence is much slower and the variability remains higher.

The **right plot** shows the distribution of the sample mean when $B = 500$. Independence (black) yields a narrow distribution with small variance; as correlation increases, the distribution widens, meaning the average fluctuates more.

This demonstrates that averaging only reduces variance effectively when the individual models or samples are **uncorrelated**. If the models are correlated (because they rely on similar features or data points) the benefit of averaging diminishes.

In the next section, we see how random forests address this limitation by introducing randomness in feature selection (by randomly choosing subsets of predictors at each split) which reduces correlation between trees and improves variance reduction beyond bagging.

```
#TODO add this foot ntoe_ the specific mechanism in random forests where, at each split of a
```

2.2 Bagging: correlated trees

- In bagging, each tree is trained on a bootstrap dataset that is drawn **with replacement** from the original training data. Because these bootstrap samples overlap substantially, the trees share **many common observations**.

- The decision tree algorithm is **greedy**: it chooses at each split the variable that gives the largest reduction in impurity. Since the datasets are very similar, the trees tend to select the **same predictors for their early splits**.

```
#TODO include this as a footnote: Impurity quantifies how mixed the node is. The decision tre
# In classification, impurity is measured using the Gini index or entropy, while in regressio
```

- As a result, the trees end up looking very similar and their predictions $\hat{f}^{*b}(x_0)$ for a new input x_0 are **highly correlated** for all $b = 1, \dots, B$. In other words, for all $b = 1, \dots, B$" means that this correlation property applies across all the trees in the bagged ensemble, each $\hat{f}^{*b}(x_0)$ tends to give a similar prediction for the same input x_0.

As seen in Figure 2, the three trees trained on different bootstrap samples from the same dataset share much of their structure.
They all split early on similar variables (Thal, Ca, Oldpeak), showing that even after resampling, the bagged trees remain strongly correlated.This shows how, despite resampling, the bagged trees remain **strongly correlated** because they repeatedly rely on the same dominant predictors for their early decisions.

Such correlation limits the benefit of averaging in bagging: even though we combine many trees, if they make **similar errors**, the ensemble's variance does not decrease as much as expected.
This motivates the next step, i.e., **random forests**, which introduce randomness in feature selection at each split to decorrelate the trees and achieve greater variance reduction.

```
#TODO but if we add randomness to then reduce the variance, isnt it a bit working twice?
```

2.3 Random forest: main idea

- **Random forests** extend the concept of bagging. The key idea is to **decorrelate** the trees by introducing randomness during the growing process.
- In bagging, trees are highly correlated because they often use the same strong predictors for their first splits. Random forests reduce this correlation by **randomly selecting a subset of predictors** at each split.
- This randomisation prevents all trees from relying on the same dominant variables, leading to a more diverse ensemble and stronger variance reduction.

2.3.1 Random forest algorithm

1. **Bootstrap the data** B times, obtaining B bootstrap samples $Z^{*1}, \dots, Z^{*B}$.
2. **Grow a tree** on each bootstrap sample. At each node, randomly select m predictors out of p, and choose the best split among those m.

3. **Grow the trees fully**, without pruning (as in bagging).
4. **Aggregate predictions**:
    - For **regression**, average the predictions from the B trees.
    - For **classification**, use the **majority vote** among the B trees.

2.3.2 Intuition and choice of m

As seen in Figure 5, selecting only a random subset of variables at each split prevents trees from becoming too similar.
Even though all trees are trained on overlapping bootstrap samples, their different variable choices make them less correlated, which in turn improves variance reduction and generalisation performance.

Typical values for m:

- For **classification**, $m \approx \sqrt{p}$.
- For **regression**, $m \approx p/3$.

This rule of thumb works well in practice because it balances diversity and predictive strength: small m increases randomness (reducing correlation), while large m allows trees to use stronger predictors (reducing bias).

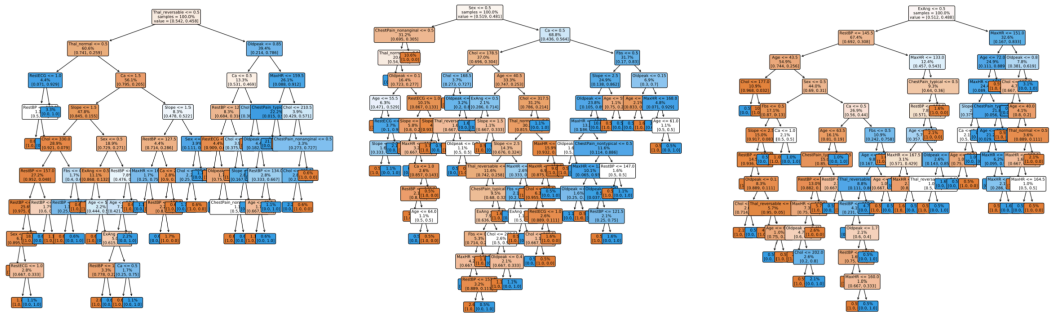

Figure 5: Random forest trees

2.4 Heart data: bagging and random forest in Python

As shown in Figure 6, the **out-of-bag (OOB) error rate** decreases rapidly as the number of trees increases for both methods. The experiment is based on the *Heart Disease* dataset, where the goal is to predict whether a patient has heart disease (AHD = Yes/No) using medical predictors such as age, cholesterol, and maximum heart rate. The code trains two ensemble models:

- A **bagging classifier**, where each tree is built from a bootstrap sample of the training data.

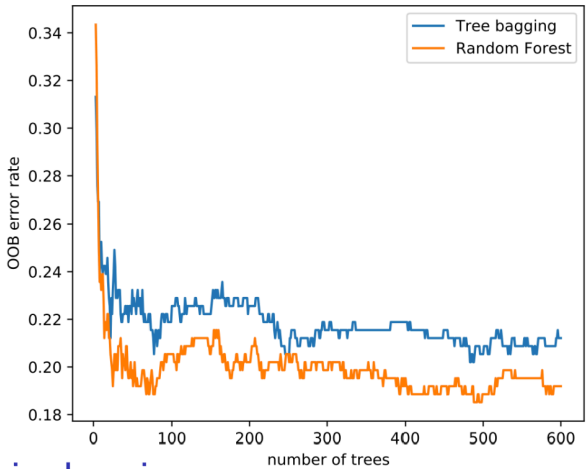

Figure 6

- A **random forest classifier**, which extends bagging by randomly selecting a subset of predictors at each split (`max_features="sqrt"`).

Each model is trained incrementally (`warm_start=True`) from 3 to 600 trees, and the **OOB score** (an internal cross-validation estimate) is tracked at every step.
The **random forest** (orange line) achieves a lower OOB error than bagging (blue line) because random feature selection reduces tree correlation, leading to lower variance and improved generalisation.
This improvement arises because random feature selection decorrelates the trees, reducing the ensemble's overall variance.

Example: Comparison of classification performance using confusion matrices. The figure on the right compares how bagging and random forest predict heart disease:
- The **top confusion matrix** corresponds to **bagging**. It correctly classifies 80% of patients without heart disease and 77% of those with it.
- The **bottom confusion matrix** corresponds to the **random forest**, which slightly improves accuracy to 83% and 78% respectively.
These results confirm that random forests enhance predictive power by combining decorrelated trees, producing more stable and generalisable predictions than simple bagging.

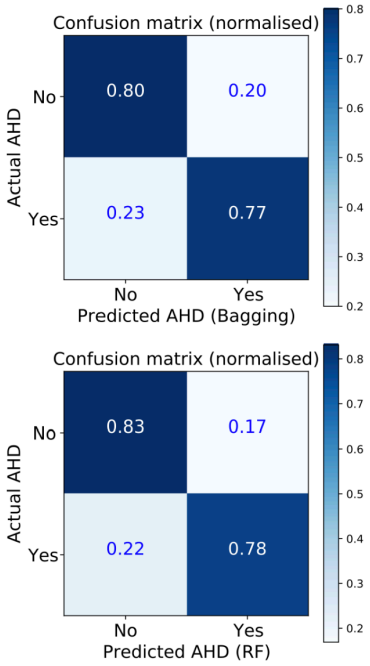

Figure 7: Confusion matrices for bagging (top) and random forest (bottom).

2.5 Variable importance

- Bagging and random forests have **high predictive power**, but the resulting ensemble of trees is difficult to interpret visually.
- To recover interpretability, we define an **overall measure of variable importance** that quantifies how much each predictor contributes to model performance.
- For **regression**, we record the total reduction in **Residual Sum of Squares (RSS)** due to splits involving a predictor X_j.
- For **classification**, we record the total decrease in **Gini index** or **deviance** associated with each predictor.
- Another way to assess importance is by **randomly permuting** a given predictor among the **out-of-bag (OOB) observations** and measuring how much the model's prediction error increases. A strong increase indicates that the variable was important for accurate predictions.

As seen in Figure 8, the variables *Thal* (thalassemia type) and *Ca* (number of major vessels) are the most influential predictors of heart disease, followed by *ChestPain* and *Oldpeak*. Features such as *Fbs* (fasting blood sugar) or *RestECG* contribute comparatively little to the model's performance.
This ranking provides a powerful interpretative tool to understand which features drive the predictive

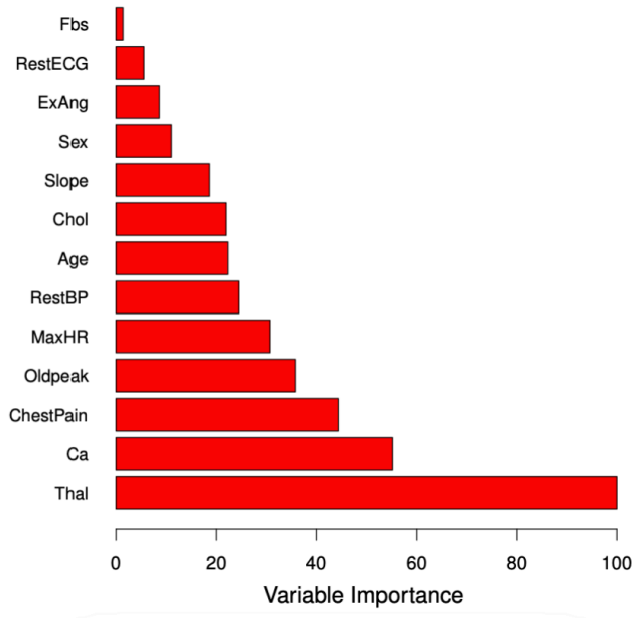

Figure 8: Variable importance in the heart disease dataset

accuracy of the random forest, even though the individual trees themselves are no longer easily interpretable.

2.6 Random forests and nearest-neighbours

Random forests can be understood as an **adaptive weighted nearest-neighbours method**, where "closeness" is defined by how often two observations fall into the same leaf nodes across trees.

- Recall that trees in a random forest are **fully grown and unpruned**. Each terminal node (leaf) represents a small region of the feature space containing similar training observations.
- For a new input x_0, each tree in the forest assigns x_0 to one of these regions, and the predicted response y_0 is the average of the y_i values within that leaf.
- Thus, for each tree, the prediction for x_0 is associated with one or more "neighbouring" training observations x_i, analogous to **nearest-neighbour methods** — except that here, *proximity is defined by the structure of the tree*, not by Euclidean distance.
- Each observation x_i contributes to the final prediction with a weight w_i proportional to how often it appears in the same terminal node as x_0 across the ensemble. Formally, the random forest prediction is

$$\hat{y}_0 = \sum_{i=1}^{n} w_i y_i, \quad \text{where } w_i \geq 0$$

and w_i represents the **fraction of trees** where x_i and x_0 fall into the same leaf.

Hence, random forests can be viewed as a **smoothing method** that adapts locally to the data.
Unlike standard k-NN, the notion of "neighbourhood" is **data-driven and complex**, determined by the hierarchical partitioning of the feature space in multiple random trees.

2.7 Pros and cons of random forests

Random forests extend bagging by introducing randomness in feature selection, resulting in strong predictive models that generalise well to unseen data.
They are highly practical because they combine **robustness**, **accuracy**, and **low tuning requirements**, making them one of the most reliable ensemble methods in practice.

Pros:

- **Excellent predictive performance:** combining many uncorrelated trees drastically reduces variance and improves accuracy on both regression and classification tasks.
- **Minimal tuning required:** only a few parameters (e.g., number of trees B, number of features m per split) need to be set, and performance is typically stable across a wide range of values.
- **Built-in validation:** the **out-of-bag (OOB) error estimate** provides an internal measure of test error, eliminating the need for cross-validation.
- **Variable importance:** random forests naturally provide a measure of which predictors contribute most to the model's performance, aiding interpretability at the feature level.
- **Ease of implementation:** readily available in standard libraries such as *scikit-learn* and *R's randomForest*, and relatively fast to train even with large datasets.

Cons:

- **Loss of interpretability:** by averaging hundreds of trees, the model becomes a "black box."
  Although we can rank variable importance, we lose the intuitive visualisation and explanation that a single decision tree offers.
  This makes random forests less suitable for contexts where transparency and explainability are essential, such as legal or medical decision-making.

2.8 Digit classification with random forest

As seen in Figure 9, the **digit classification** task involves recognising handwritten digits (0–9) based on pixel intensity values.
Each model is trained on the same dataset and evaluated on its ability to correctly classify unseen digits.

Figure 9: Examples of handwritten digits used in the classification task.

Models compared:

- **linreg:** direct linear regression.
- **LDA1:** Linear Discriminant Analysis (LDA) based on the values x_i.
- **LDA2:** LDA based on x_i and x_i^2.
- **QDA:** Quadratic Discriminant Analysis with a distinct covariance matrix for each class.
- **log:** logistic regression (multinomial).
- **log-ridge:** logistic regression with ridge penalty.
- **log-lasso:** logistic regression with lasso penalty.
- **RF:** random forest.

Training and test errors (%)

Method	Training	Test
linreg	7.6	13.1
LDA1	6.2	11.5
LDA2	3.9	10.2
QDA	1.8	13.5
log	0.01	11.1
log-ridge	4.1	9.0
log-lasso	2.7	8.8
RF	0.0	5.9

The results show that **random forests** achieve the lowest **test error (5.9%)**, outperforming all other methods.
This strong generalisation comes from combining many decision trees trained on bootstrapped samples with random subsets of features. thereby creating a model structure that captures **non-linear dependencies** and **complex feature interactions** across the pixel space.

This strong generalisation arises from combining many decision trees trained on bootstrapped samples with random feature subsets, creating a structure that captures **non-linear dependencies** and **complex feature interactions** across the pixel space.

Regularised logistic models (ridge and lasso) also perform well, as they prevent overfitting by penalising large coefficients.
However, random forests remain the most powerful approach here, thanks to their ability to model intricate decision boundaries while maintaining robustness through averaging and decorrelation.

2.9 Digit classification: variable importance

As shown in Figure 10, random forests allow us to interpret complex models by computing a measure of **variable importance**, in this case, identifying which **pixels** contribute most to distinguishing each handwritten digit.

Each heatmap corresponds to one digit (0–9), where the **dark regions** highlight the pixels most influential in the model's decisions.
For example, the central pixels are highly important for distinguishing **0** and **8**, whereas the **vertical strokes** dominate the importance maps for **1**.

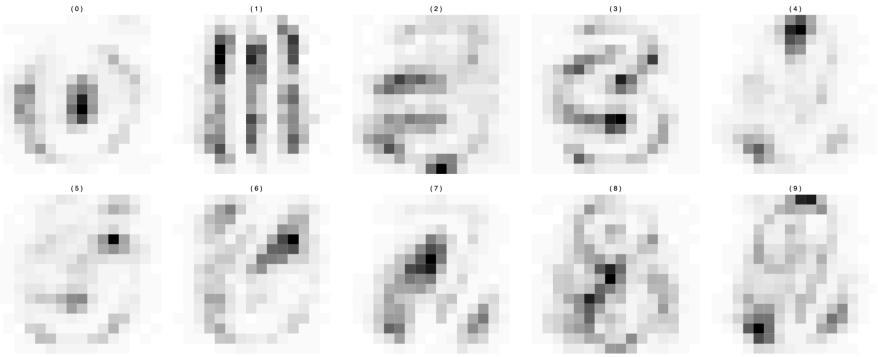

Figure 10: Pixel-level variable importance for each digit in the random forest model. Darker areas indicate higher importance.

These visual patterns align with how humans recognise digits, showing that the model learns to focus on the structural areas that define each number.

This technique provides a useful diagnostic tool:

- It helps confirm that the random forest has learned meaningful visual features rather than random noise.
- It also offers limited interpretability despite the ensemble's overall "black-box" nature, giving insights into what drives the model's predictions.

Thus, pixel-level variable importance visualisation bridges the gap between **performance** and **explainability**, showing how random forests internally differentiate complex visual classes.

3 Boosting

3.1 Boosting for trees

Boosting builds upon the idea of bagging but reverses its focus. In **bagging**, we use many **unbiased**, high-variance models (deep trees) and reduce their variance through averaging. Each tree is grown independently on a bootstrap sample[2] of the data, and their predictions are averaged to form a stable, low-variance model.

[2] A bootstrap sample is a dataset obtained by **sampling with replacement** from the original training data. Each decision tree in bagging is trained on a different bootstrap sample, which introduces randomness: although all trees see roughly the same number of observations, they each **contain different subsets and repetitions** of the data. This variability makes the trees slightly different from one another, allowing their predictions to be **averaged** to reduce variance in the final ensemble. Averaging smooths out these fluctuations, it reduces variance, but does not change the underlying model's bias (each tree is still an unbiased estimator, just noisy).

Boosting, in contrast, aims to reduce the **bias** of the model. Instead of combining many deep, independent trees, it combines many **weak**, biased learners (typically shallow trees) that individually have low variance. These trees are not grown independently but **sequentially**: each new tree focuses on the parts of the data that the previous ones predicted poorly.

In other words:

- Bagging: reduces **variance** by averaging large, independent trees.
- Boosting: reduces **bias** by adding small trees that correct previous errors.
- Each tree in boosting learns from the **residuals** of the preceding model, gradually improving performance.
- This sequential correction makes boosting a form of *iterative learning*, where each step targets what remains unexplained.

3.2 Boosting algorithm for regression trees

The boosting algorithm formalises this process for regression trees. It builds the model step by step, with each new tree improving upon the errors of the last. Before starting, we fix three tuning parameters:

- **Number of repetitions** $B \in \mathbb{N}$: the number of trees (iterations).
- **Number of splits** $d \in \mathbb{N}$: the depth of each tree, controlling its complexity.
- **Shrinkage parameter** $\lambda \in (0, 1)$: a learning rate that controls how strongly each tree contributes to the model. $\lambda = 1$ no shrinkage; fast learning but high risk of overfitting. With λ too close to 0, learning becomes very slow, as each tree makes only a tiny adjustment.

Given a training set (x_i, y_i), $i = 1, \dots, n$, the algorithm proceeds as follows:

1. **Initialisation**: start with a null model and set the initial residuals to the observed outcomes:

$$\hat{f}(x) = 0, \quad r_i = y_i.$$

   This means the model initially predicts zero for all observations. So the residuals are numerically equal to the observed responses y_i, because the model hasn't learned anything yet. That's why the first residuals are sometimes described as the original response values or the real observations.

2. **Iterative fitting** For $b = 1, \dots, B$:

   - Fit a regression tree $\hat{f}^b$ with d splits to the data (x_i, r_i). Each tree now tries to explain the residuals: the part of y_i that remains unexplained by the model so far.
   - Update the model by adding a *shrunken* version of this tree:

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x).$$

     The shrinkage factor λ prevents overfitting by slowing down the learning process.

- Update the residuals to reflect the remaining error after adding the new tree:

$$r_i \leftarrow r_i - \lambda \hat{f}^b(x_i).$$

Residuals that remain large indicate regions where the model still performs poorly, guiding the next tree to focus on those points.

3. **Final model** After B iterations, the boosted model is the sum of all the shrunken trees:

$$\hat{f}(x) = \sum_{b=1}^{B} \lambda \hat{f}^b(x).$$

This model captures complex patterns by successively correcting its own mistakes — an effect that makes boosting powerful, yet prone to overfitting if B is too large.

In summary, boosting performs **slow learning** through small, sequential updates. While each tree alone is a poor predictor, together they form a strong model that reduces bias and achieves high accuracy when the tuning parameters are chosen carefully.

3.3 Tuning and learning rate in boosting

Boosting can be understood as a form of **slow learning**, since it builds the model step by step using many small trees that gradually improve $\hat{f}$ in regions where it performs poorly. Each tree makes only a small correction to the current model, focusing on residuals that remain large. Over successive iterations, these corrections accumulate into a powerful ensemble.

The **shrinkage parameter** λ further slows this process, controlling how much each new tree contributes. A smaller λ makes learning more gradual, allowing the algorithm to fit a wider variety of tree shapes to the residuals and avoid overfitting. This mechanism ensures that improvement happens progressively and in diverse parts of the predictor space.

Boosting has three key **tuning parameters**:

- **(a) Number of trees** B: determines how many iterations the algorithm performs.
  Unlike bagging and random forests, boosting can overfit if B becomes too large, although this typically happens slowly. The optimal B is chosen via cross-validation to stop learning before overfitting begins.
- **(b) Shrinkage parameter** λ: typically a small positive number (e.g. 0.01 or 0.001).
  It controls the *learning rate* of the algorithm. Smaller values imply slower learning but often lead to better generalisation, requiring more trees for good performance.
- **(c) Number of splits** d: controls the complexity of each individual tree.
  When $d = 1$, each tree is a **stump**, i.e. a single split. Such simple trees usually work surprisingly well because boosting reduces bias sequentially, making deep trees unnecessary.

Boosting can also be applied to **classification** problems, although the algorithm's formulation is slightly more complex than for regression.

3.4 Hitters data: boosting

To illustrate the behaviour of boosting, consider its application to the **Hitters** dataset, where the goal is to predict baseball players' salaries. The plots below show the **test error** as a function of the number of boosting iterations B for different values of the shrinkage parameter λ and tree depth d.

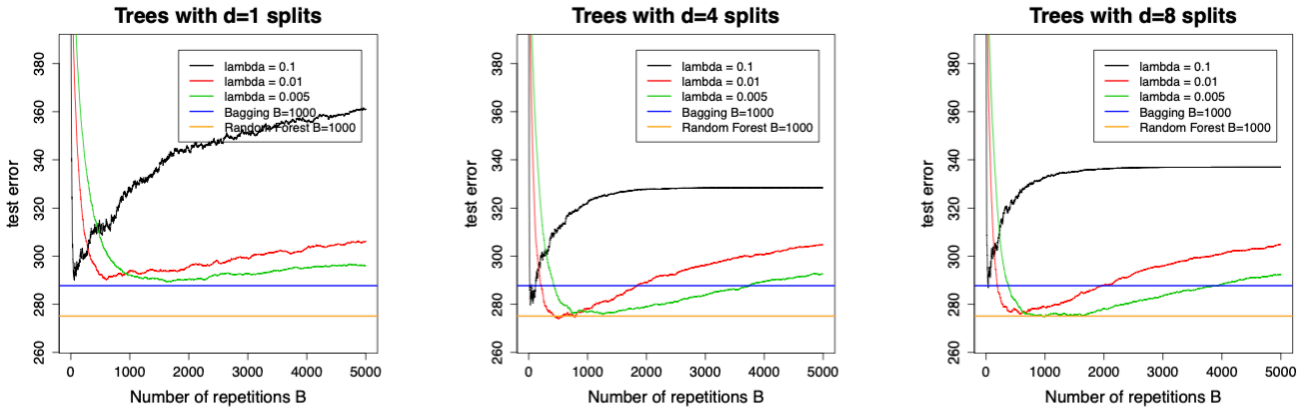

Figure 11: Test error as a function of the number of boosting iterations B for different values of λ and tree depths $d = 1, 4, 8$.

When $\lambda = 0.1$, the model learns very quickly, and the test error drops sharply but also rises again soon, which is a sign of **overfitting**. A small λ (e.g. 0.01 or 0.005) slows learning, resulting in a more gradual decrease of the test error that remains low over a broader range of B. This makes model selection via cross-validation more stable.

With **stumps** ($d = 1$), the model initially underfits, but increasing the depth to $d = 4$ allows it to reach lower test errors comparable to those of bagging and random forests. Further increasing d (e.g. to 8) offers little benefit and can even worsen performance, as overfitting begins earlier.

In practice, a **small learning rate** (around 0.01) and **moderately shallow trees** (e.g. $d = 4$) tend to yield the best trade-off between bias reduction and generalisation. Boosting thus provides a flexible yet controlled way to improve predictive performance through slow, sequential learning.

4 Support Vector Machines: Maximal Margin Classifier

Support Vector Machines (SVM) represent a geometric approach to classification. Unlike LDA or logistic regression, which model statistical relationships between Y and X, SVMs make **no distri-**

butional assumptions. They aim instead to find a **decision boundary** (a hyperplane) that separates classes in the feature space.

A **hyperplane** in $\mathbb{R}^p$ is defined as the set of all x satisfying the linear equation

$$\beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p = 0.$$

This boundary divides the predictor space into two half-spaces. For any observation z, the function $z = \beta_0 + \beta_1 z_1 + \cdots + \beta_p z_p$ indicates its position relative to the hyperplane: points with $f(z) > 0$ belong to one class, and those with $f(z) < 0$ to the other. Points exactly on the hyperplane satisfy $f(z) = 0$.

The vector $\beta = (\beta_1, \dots, \beta_p)$ is called the **normal vector** because it is orthogonal to the hyperplane. Its direction determines the orientation of the separating boundary, and its length is usually normalised to one to ensure a unique parametrisation. The intercept β_0 shifts the hyperplane along the direction of β without changing its orientation.

To build intuition:

- Points with $|f(z)|$ large are far from the boundary.
- Points with $f(z) \approx 0$ lie close to it and are more uncertain.
- Thus, the magnitude $|f(z)|$ can be interpreted as the **distance** from z to the hyperplane.

Now consider a binary classification problem with classes labelled $y_i \in \{-1, +1\}$.
A hyperplane **separates** the training data if it assigns positive values of $f(x_i)$ to all observations with $y_i = +1$ and negative values to all with $y_i = -1$. Formally,

$$y_i(\beta_0 + \beta^\top x_i) > 0, \quad \text{for all } i = 1, \dots, n.$$

If such a hyperplane exists, there are infinitely many possible ones, each perfectly separating the two classes but differing in position.

The **Maximal Margin Classifier (MMC)** resolves this ambiguity by choosing the separating hyperplane that **maximises** the **margin**, defined as the smallest distance between the hyperplane and any training observation. Denoting this distance by M, the optimisation problem is:

$$\max_{\beta_0, \beta, M} M \quad \text{s.t.} \quad \|\beta\| = 1,\ y_i(\beta_0 + \beta^\top x_i) \geq M \text{ for all } i.$$

This ensures all observations lie at least M units away from the separating boundary, yielding the widest possible margin.

The observations that lie exactly at this minimal distance, i.e., those that satisfy $y_i(\beta_0 + \beta^\top x_i) = M$, called the **support vectors**. They define the position of the maximal margin hyperplane. Small changes to these points can shift the boundary, whereas non-support points have no effect.

Although conceptually elegant, the MMC has two key **limitations**:

1. It is **unstable**, as it depends heavily on the few support vectors near the boundary.
2. It only works when the classes are **linearly separable**. In most real-world data, perfect separation is impossible, motivating more flexible extensions such as the **Support Vector Classifier**, which allows for a *soft* margin.

SVMs therefore build upon this geometric idea, gradually introducing relaxation and nonlinearity to handle more complex data structures while maintaining the same underlying optimisation principle.

4.1 Hyperplanes: construction

A **hyperplane** in $\mathbb{R}^p$ is defined by the equation

$$\beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p = 0.$$

This represents a $(p-1)$-dimensional flat surface that divides the feature space into two half-spaces.

The vector $\beta = (\beta_1, \ldots, \beta_p)$ is called the **normal vector**, as it is **orthogonal** to the hyperplane. Its direction indicates the orientation of the separating boundary, while the intercept β_0 determines its distance from the origin.

To ensure a unique parametrisation, we usually normalise the normal vector so that $\|\beta\| = 1$. Without this constraint, scaling both β_0 and β by the same constant would produce the same hyperplane but different parameter values.

For any point $z \in R^p$, the signed distance from the hyperplane is given by $|\beta_0 + \beta_1 z_1 + \cdots + \beta_p z_p|$.
- If this value is **positive**, z lies on one side of the hyperplane.
- If it is **negative**, z lies on the opposite side.
- If it equals **zero**, z lies exactly on the hyperplane.

As shown in Figure 12, the **blue line** represents the hyperplane in $\mathbb{R}^2$, while the **red arrow** depicts the normal vector that is orthogonal to it. The absolute value of the linear function corresponds to the **distance** between each point and the hyperplane, capturing how far an observation lies from the separating boundary.

4.2 Separating hyperplanes

We now focus on the simplest SVM setting: two classes, denoted by $\mathcal{G} = \{-1, +1\}$.
Given training data $(x_1, y_1), \ldots, (x_n, y_n)$ with $y_i \in \{-1, +1\}$, we say that a **hyperplane is separating** if it correctly classifies all observations according to

$$\beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip} > 0 \text{ if } y_i = +1, \quad \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip} < 0 \text{ if } y_i = -1.$$

These two inequalities can be written more compactly as

$$y_i(\beta_0 + \beta^\top x_i) > 0, \quad i = 1, \ldots, n.$$

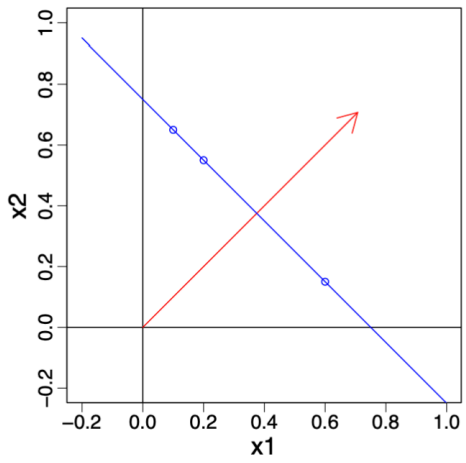

Figure 12: A hyperplane (**blue**) in $\mathbb{R}^2$ with its normal vector (**red**) orthogonal to it. The absolute value of the linear function gives the distance of each point from the hyperplane.

This expression captures both cases simultaneously: if $y_i = +1$, the term remains positive, while if $y_i = -1$, it reverses sign to ensure that the point lies on the opposite side of the hyperplane.

A separating hyperplane therefore divides the feature space so that all points of one class lie on one side and all points of the other class lie on the opposite side. When such a hyperplane exists, there are **infinitely many** possible separating boundaries, each producing zero training error but potentially different generalisation properties.

As illustrated in Figure 13, several hyperplanes can perfectly separate the same data. The plot on the left shows multiple valid separating lines, all achieving perfect classification, while the right panel depicts one such decision boundary dividing the feature space into two regions corresponding to the two classes.

4.3 Maximal Margin Classifier

When a separating hyperplane exists, it defines a **natural classifier** for new data points. The function $f(x_0)$ tells us on which side of the hyperplane a point x_0 lies: if it is positive, we assign the class $+1$, and if negative, the class -1. The **magnitude** of this value indicates how far the point lies from the boundary.

The **margin** represents the smallest distance between the training data and the separating hyperplane. The **Maximal Margin Classifier (MMC)** selects the hyperplane that maximises this margin, thereby creating the **widest possible gap** between the two classes. A wider margin leads to a more robust classifier, as small changes in the data are less likely to alter the decision boundary.

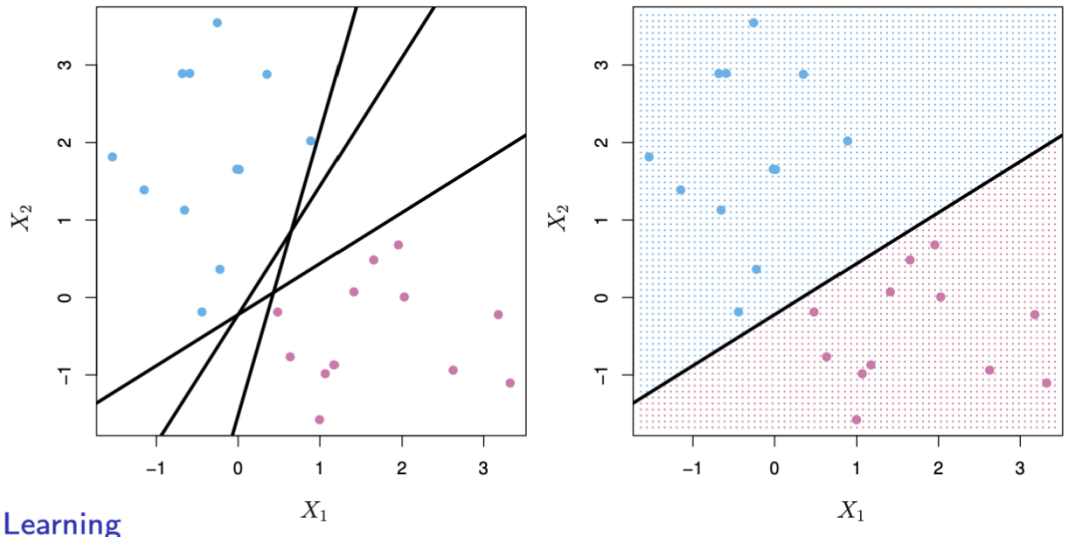

Figure 13: Different possible separating hyperplanes for linearly separable data.

Only the observations lying exactly on the margin determine the boundary. These are known as **support vectors** because they "hold" the hyperplane in place. If one of them moves, the hyperplane shifts as well. Points further from the margin have no influence on its position. The MMC is obtained by solving a convex quadratic optimisation problem that efficiently identifies the separating hyperplane with the largest margin.

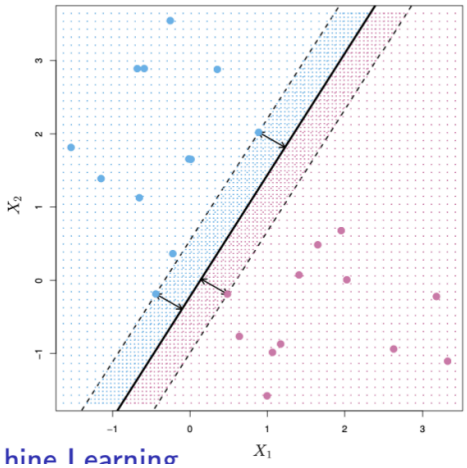

Figure 14: MMC: the optimal hyperplane (solid line) maximises the distance to the nearest training points of either class, defining the margin (dashed lines).

4.4 Comments and limitations

The optimisation problem behind the **maximal margin classifier** is a **convex quadratic programme**, which guarantees that the global optimum can be found efficiently.

However, as seen in Figure 14, only a few observations (the **support vectors**) determine the position of the separating hyperplane. Slightly moving one of these support vectors alters the optimal boundary, while moving other observations has no effect at all. This dependence on just a few critical points makes the classifier **sensitive** and potentially **unstable**.

In practice, this instability is compounded by another limitation: **most datasets are not perfectly linearly separable**. As illustrated in Figure 14, the maximal margin classifier only works when a hyperplane can fully separate the classes. When such perfect separation is impossible, the optimisation problem has no feasible solution.

These challenges motivate the next step, i.e., the **Support Vector Classifier**, which introduces a *soft margin* allowing for some classification errors while maintaining the geometric intuition of maximising the margin.

5 Support Vector Machines: Support Vector Classifier

5.1 Support Vector Classifier

The maximal margin classifier seeks a separating hyperplane that classifies the training data perfectly, which is often too rigid in the presence of noise or mislabelled observations. The support vector classifier is a *soft margin* method that permits controlled violations of the margin to improve robustness and generalisation.

We introduce slack variables $\xi_i \geq 0$ for training observations (x_i, y_i) with $y_i \in \{-1, +1\}$ and solve the optimisation problem:

$$\min_{\beta_0, \beta, \xi} \left(\frac{1}{2}\|\beta\|^2 + C \sum_{i=1}^{n} \xi_i \right) \quad \text{s.t.} \quad y_i(\beta_0 + \beta^\top x_i) \geq 1 - \xi_i,\ \xi_i \geq 0.$$

Here, the parameter $C > 0$ controls the trade-off between a wide margin (small $\|\beta\|$) and tolerance for violations (large ξ_i). In other words, the parameter C determines how strongly the model penalises violations of the margin (i.e., large ξ_i values). The variables ξ_i do not set the margin; instead, they measure how much each observation violates the margin constraint.
C controls *tolerance*, ξ_i measures *actual violations*, and β determines the *margin width*.

The positions of points are as follows:

- **Points with** $\xi_i = 0$: on or outside the margin.

- **Points with** $0 < \xi_i \leq 1$: inside the margin but correctly classified.
- **Points with** $\xi_i > 1$: misclassified.

As $C \to \infty$, we recover a hard margin (when separable). A smaller C widens the margin and typically yields a smoother, more stable decision rule focused on the most informative points — the *support vectors*.

Algorithmically:

1. Choose C (for instance, via cross-validation).
2. Solve the convex quadratic programme above.
3. Form the decision function $\hat{f}(x) = \text{sign}(\beta_0 + \beta^\top x)$, supported by training points with non-zero Lagrange multipliers (the support vectors).

Interpretation: the classifier allocates a *budget* of margin violations to gain robustness to outliers, achieve better average classification, and admit solutions when the classes are not perfectly separable.

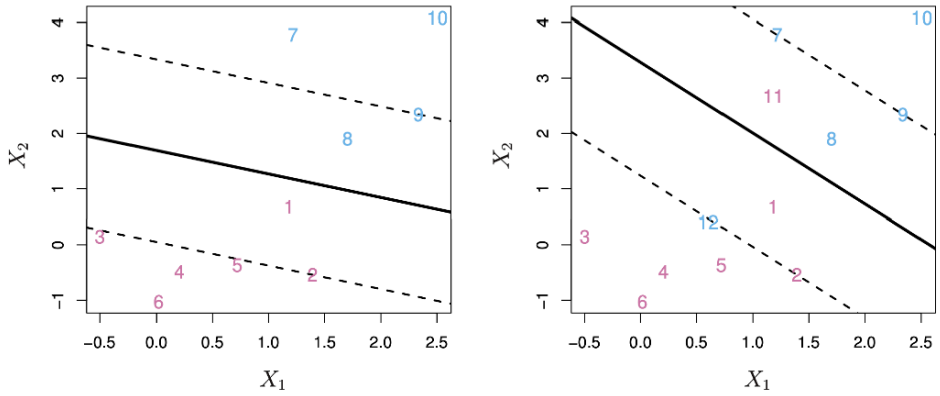

Figure 15: Hard- versus soft-margin solutions on the same dataset.

As seen in Figure 15, the **left-hand plot** shows the *maximal margin classifier*, which fits the training data perfectly but results in a narrow margin that is highly sensitive to outliers. Even one mislabelled point could completely change the orientation of the separating hyperplane.

The **right-hand plot** depicts the *support vector classifier*, where some points are allowed to lie inside or even beyond the margin (those with $\xi_i > 0$). This flexibility leads to a wider margin and a more stable decision boundary.
The support vectors (the points closest to or inside the margin) determine this boundary, while all other points play no direct role in its position.

5.2 Mathematical formulation of the support vector classifier

The soft-margin formulation can also be expressed as a maximisation problem. We aim to find the largest possible margin M while allowing controlled violations through slack variables $\xi_i \geq 0$.

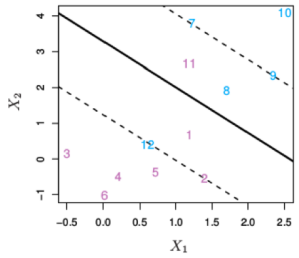

Figure 16: Soft-margin classifier with slack variables ξ_i and total budget b.

The optimisation problem is:

$$\max_{\beta_0,\ldots,\beta_p,\ \xi_1,\ldots,\xi_n} M$$

subject to

$$\sum_{j=1}^{p} \beta_j^2 = 1, \quad y_i(\beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip}) \geq M(1 - \xi_i),$$

$$\xi_i \geq 0, \quad \sum_{i=1}^{n} \xi_i \leq b.$$

As illustrated in Figure 16, the soft-margin classifier allows some points to fall inside or even beyond the margin, depending on their corresponding ξ_i values.
Each ξ_i quantifies how much observation i violates the margin constraint:

- $\xi_i = 0$: the observation lies on or outside the margin (correctly classified).
- $0 < \xi_i \leq 1$: the observation lies inside the margin but is still correctly classified.
- $\xi_i > 1$: the observation is misclassified, on the wrong side of the separating hyperplane.

In Figure 16, the points exactly on the margin correspond to $\xi_i = 0$, while those inside the shaded margin regions have $0 < \xi_i \leq 1$.
Points that cross the decision boundary entirely (on the wrong side) represent $\xi_i > 1$ and contribute most to the violation term in the optimisation.

The constant $b > 0$ acts as a *tuning* or *regularisation* parameter that sets the total "budget" of margin violations across all n observations. A larger b allows more flexibility, useful for noisy or non-separable data, while a smaller b enforces stricter separation with fewer violations.

Here, the *support vectors* are the observations that either lie exactly on the margin or violate it (i.e. those for which $\xi_i > 0$). Only these points determine the position and orientation of the separating hyperplane ; all other observations, being further from the decision boundary, have no influence on the classifier.

Consequently, the regularisation parameter b indirectly controls the number of support vectors and hence the complexity of the model. A smaller b (stricter margin) limits violations, reducing the number of support vectors and yielding a simpler, higher-bias but lower-variance model. Conversely, a larger b allows more margin violations, increasing the number of support vectors and producing a more flexible, lower-bias but higher-variance classifier.

5.3 Tuning parameter b

The regularisation constant b plays a central role in controlling the flexibility of the support vector classifier. It sets the total "budget" of margin violations permitted across all training observations, and therefore determines the number of support vectors used to define the decision boundary.

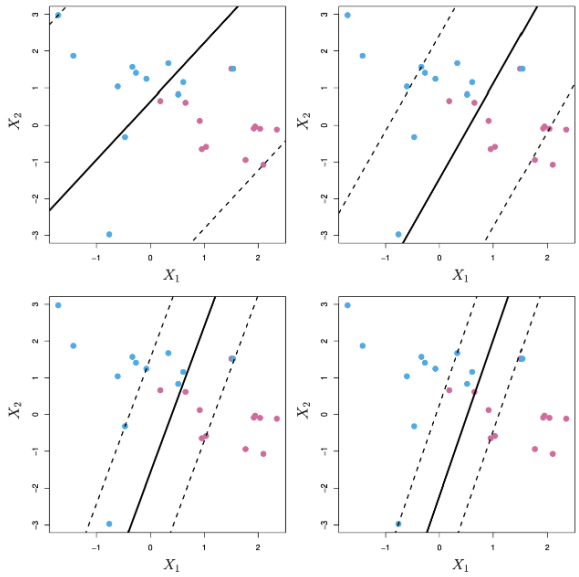

Figure 17: Effect of the tuning parameter b on the margin width and classifier flexibility.

As illustrated in Figure Figure 17, increasing or decreasing b directly influences the width of the margin and the bias–variance characteristics of the model:

- **Large** b allows more violations, leading to **wider margins** and a **higher-bias, lower-variance** classifier. The decision boundary becomes smoother and less sensitive to individual observations.
- **Small** b enforces fewer violations, producing **tighter margins** and a **lower-bias, higher-variance** classifier. The boundary fits the training data more closely but risks overfitting.

The optimal value of b is typically chosen through **cross-validation**, balancing the trade-off between margin width and classification accuracy on unseen data.

5.4 Limitations of the support vector classifier

Although the support vector classifier is a powerful method for constructing linear decision boundaries, it has an important limitation: it can only produce *linear* separation in the feature space.

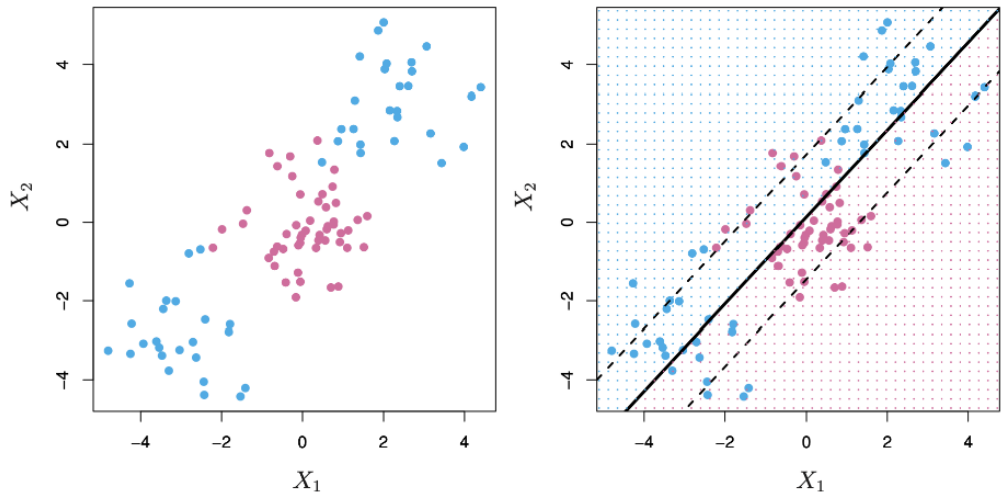

Figure 18: Limitation of the support vector classifier: poor performance with non-linear class boundaries.

As shown in Figure 18, when the true decision boundary is non-linear, a linear classifier (such as the support vector classifier) fails to capture the complex shape that separates the two classes. In such cases:

- A **linear decision boundary** performs poorly if the data are not linearly separable in the feature space $(X_1, \dots, X_p)$.
- One might try to **enlarge the feature space**, for example by adding polynomial terms such as $X_1^2, X_2^2, \dots$, and then apply a linear classifier in this higher-dimensional space.
- However, enlarging the feature space dramatically increases dimensionality, and the resulting **computational cost** can become infeasible for large datasets.

This limitation motivates the next step: introducing **non-linear decision boundaries** through the use of kernels, which implicitly map the data into a higher-dimensional space without the need to compute the expanded features explicitly.

6 Support Vector Machines

6.1 Support Vector Machines

Support Vector Machines (SVMs) generalise the support vector classifier to allow **non-linear decision boundaries**. They achieve this by enlarging the feature space — but in an *automatic* way, without explicitly constructing new variables as in manual feature engineering.

In the support vector classifier, we used the decision function

$$f(x) = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p,$$

which defines a **linear hyperplane** in the feature space. Once the optimal parameters are found, this function is used to classify new observations:

$$\hat{G}(x) = \begin{cases} +1 & \text{if } f(x) > 0, \\ -1 & \text{if } f(x) < 0. \end{cases}$$

A deep mathematical result shows that the optimal separating function can be rewritten as

$$f(x) = \beta_0 + \sum_{i=1}^{n} \alpha_i y_i \langle x, x_i \rangle,$$

where each α_i is a parameter associated with training observation i, and $\langle x, z \rangle = \sum_{j=1}^{p} x_j z_j$ denotes the **inner product** between two feature vectors.

Only a small subset of training points have $\alpha_i \neq 0$; these correspond to the **support vectors** that determine the decision boundary. All other points do not influence the classifier.

The inner product $\langle x, z \rangle$ can be interpreted as a **similarity measure**: it is large when two vectors point in similar directions (highly correlated) and small when they are nearly orthogonal. This observation motivates the idea of **replacing** the inner product with a more general similarity function $K(x, z)$, called a **kernel**.

By doing so, the Support Vector Machine retains the same structure as the support vector classifier but gains the ability to create **non-linear decision boundaries** in the original predictor space.

6.2 The Kernel Trick

Support Vector Machines classify observations using a function that is, in general, *non-linear* in the original feature space:

$$f(x) = \beta_0 + \sum_{i=1}^{n} \alpha_i y_i K(x, x_i),$$

where $K(x, z)$ is a *(positive definite)* **kernel function** measuring the similarity between two feature vectors $x, z \in \mathbb{R}^p$.

By replacing the inner product $\langle x, z \rangle$ with a kernel $K(x, z)$, the SVM effectively **enlarges the feature space implicitly**, without ever computing the higher-dimensional representation explicitly.
This elegant idea is known as the **kernel trick**.

- With the **linear kernel**, $K(x, z) = \langle x, z \rangle$, we recover the standard support vector classifier.
- With **non-linear kernels**, the classifier defines curved and flexible decision boundaries in the original predictor space while remaining linear in the (implicit) high-dimensional feature space.

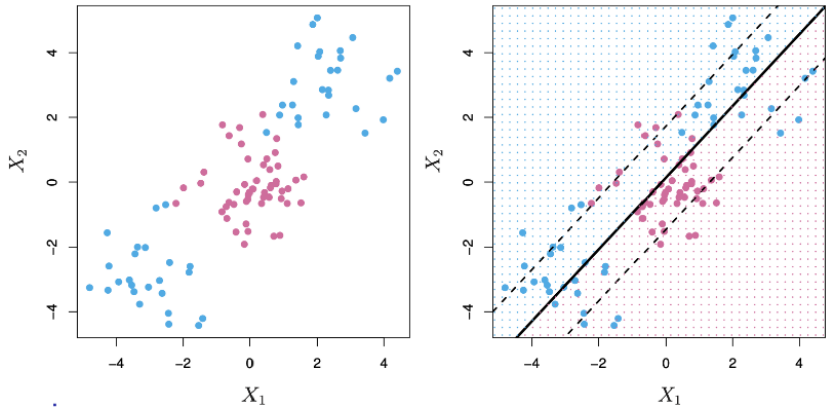

Figure 19: Illustration of the kernel trick: the SVM implicitly enlarges the feature space to achieve non-linear decision boundaries.

As seen in Figure 19, mapping the data into a higher-dimensional space allows a simple linear boundary in that space to correspond to a **non-linear boundary** in the original feature space, i.e., the essence of the kernel trick.

6.3 The Kernel Trick: Examples

Support Vector Machines can use different kernels to implicitly map the data into higher (even infinite) dimensional spaces, enabling highly flexible decision boundaries.

6.3.1 The Polynomial Kernel

The **polynomial kernel** is defined as

$$K(x, z) = \left(1 + \gamma \sum_{j=1}^{p} x_j z_j\right)^d,$$

where γ is a scaling parameter and d is the degree of the polynomial.

This kernel corresponds to fitting a **support vector classifier in a higher-dimensional space** that includes all polynomial terms up to degree d.
In the code, this is specified as `svm = SVC(kernel='poly', degree=d, gamma=gamma)`.
The parameter γ controls the **scale** of the transformation and hence the flexibility of the decision boundary.

- Larger γ and higher d produce more flexible, curved boundaries that adapt to complex patterns.
- Smaller γ or lower d result in smoother, more regular decision surfaces.

6.3.2 The Radial (Gaussian) Kernel

The **radial basis function (RBF)** or **Gaussian kernel** is defined as

$$K(x, z) = \exp\left(-\gamma \sum_{j=1}^{p} (x_j - z_j)^2\right),$$

where $\gamma > 0$ determines how quickly similarity decays with distance.

This kernel corresponds to fitting a **support vector classifier in an infinite-dimensional feature space**, since the associated basis functions are infinitely many.
In the code, it is specified with `kernel='rbf'` inside `svm = SVC(kernel='rbf', gamma=gamma)`, allowing γ to control how local or global the decision surface becomes.

- Large values of γ make the kernel highly localised: only nearby observations influence the classification, producing **flexible, non-linear** decision boundaries.
- Small values of γ yield smoother, more global fits, where distant points still contribute, resulting in **wider margins** and more stable boundaries.

As seen in Figure 20, the **left-hand plot** based on the polynomial kernel captures some curvature but remains relatively smooth, while the **right-hand plot**, using the radial kernel, displays a highly adaptive boundary that separates non-linear patterns effectively. This demonstrates how the choice of kernel and the tuning of γ determine the **locality** and **flexibility** of the Support Vector Machine.

6.4 Comments on Support Vector Machines

Support Vector Machines (SVMs) are a **powerful and flexible tool** for classification, capable of learning both linear and non-linear decision boundaries through the use of kernels.

Although the underlying mathematics are complex, the **kernel trick** allows us to apply SVMs without needing to explicitly compute the high-dimensional feature mappings. In practice, fitting an SVM simply requires computing the pairwise similarities

$$K(x_i, x_j)$$

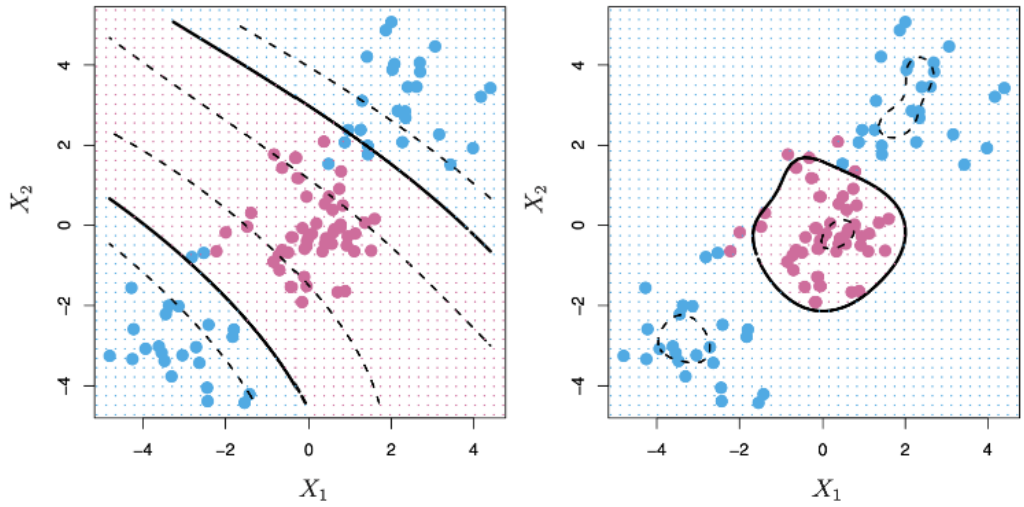

Figure 20: Polynomial and radial kernels producing different decision boundaries.

for all distinct pairs of training observations (i, j).

A wide variety of kernels have been developed, but even the **standard choices**, such as the linear, polynomial, and radial basis (Gaussian) kernels, typically perform very well across a broad range of applications.

For **multi-class problems**, SVMs can be extended using:

- **One-versus-one classification**, where a separate classifier is trained for each pair of classes; or
- **One-versus-all classification**, where each classifier distinguishes one class from all the others.

For further discussion and examples, see Chapter 9 of *An Introduction to Statistical Learning (ISL)*.

6.5 Support Vector Machines in Python

The following Python example fits Support Vector Machines on the *Iris* dataset using an **RBF (radial basis function) kernel**. Only the first two features ("sepal length" and "sepal width") are used so that the resulting decision boundaries can be visualised in two dimensions.

The code iterates over three values of the tuning parameter γ (0.1, 1, and 20) each time fitting an SVM model and plotting its resulting decision boundary.

In the code, the line `svm = SVC(C=1, kernel='rbf', gamma=gamma)` creates an SVM classifier with the specified γ, and the subsequent loop visualises the corresponding decision boundaries.

Each subplot in Figure 21 illustrates how γ controls the **locality** and **flexibility** of the model:

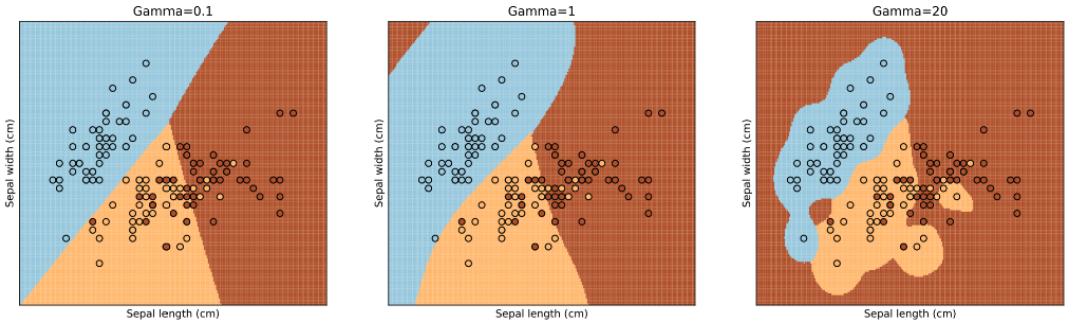

Figure 21: Decision boundaries of radial-kernel SVMs on the Iris dataset for different γ values.

- In the **left-hand plot** ($\gamma = 0.1$), the kernel is *broad and global*: similarities decay slowly, so many training points influence each decision. The resulting boundary is smooth and less adaptive, i.e., a **high-bias, low-variance** model.
- In the **middle plot** ($\gamma = 1$), the decision surface adapts more closely to the data while preserving a reasonable margin. This corresponds to a **balanced bias–variance trade-off**.
- In the **right-hand plot** ($\gamma = 20$), the kernel is *highly localised*: only nearby points affect predictions. The decision boundary becomes extremely wiggly, indicating **low bias but high variance** and the risk of overfitting.

This example highlights the **bias–variance trade-off** governed by γ: smaller values yield smoother, more generalisable boundaries, whereas larger values produce complex, locally adaptive ones.

6.5.1 Effect of the Cost Parameter

In the next example, the SVM is again fitted on the *Iris* dataset using the **radial basis function (RBF) kernel**, this time fixing $\gamma = 20$ and varying the **cost parameter** C (the inverse of the margin budget).

The parameter C determines how strongly misclassifications are penalised during training:

- **Small** C allows more margin violations (a *larger budget*) leading to a smoother, more regularised decision boundary.
- **Large** C penalises violations heavily (a *smaller budget*) yielding a tighter boundary that fits the training data more closely.

As seen in Figure 22, the **left-hand plot** with $C = 0.1$ results in a wider margin that tolerates a few misclassified points but generalises better. In contrast, the **right-hand plot** with $C = 20$ shows a much tighter boundary that fits almost all training data points correctly but risks overfitting.

This again illustrates the **bias–variance trade-off**: smaller C increases bias but reduces variance, while larger C decreases bias at the expense of higher variance.

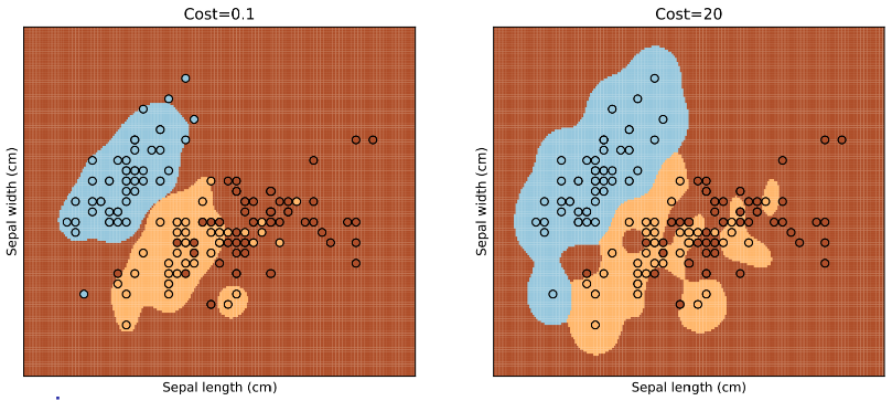

Figure 22: Decision boundaries for different values of the cost parameter C.

6.5.2 Model Selection by Cross-validation

In this final example, an SVM with a **radial basis function (RBF) kernel** is tuned automatically using **cross-validation**.
Instead of choosing the parameters γ and C manually, a grid search is performed over a predefined range of values, and the model is evaluated with k-fold cross-validation to identify the best-performing combination.

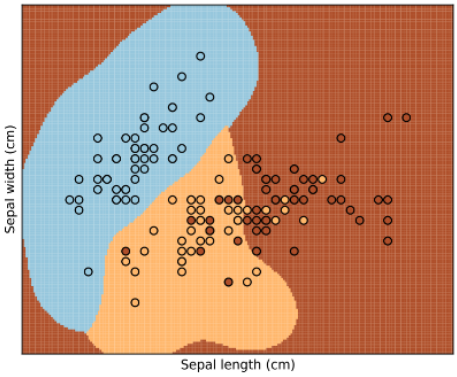

Figure 23: Optimal decision boundary obtained by tuning the parameters C and γ via cross-validation.

As illustrated in Figure 23, the selected parameters ($C = 1.2$, $\gamma = 5$ in this case) yield a decision boundary that balances **flexibility** and **smoothness**. Compared with the earlier examples:

- The boundary is less rigid than with a small γ or small C, avoiding underfitting.
- It is also less wiggly than with excessively large values, preventing overfitting.

This process demonstrates the value of **cross-validation** for model selection: it systematically explores

combinations of hyperparameters to achieve the best trade-off between bias and variance, ensuring robust generalisation on unseen data.

6.6 Digit Classification with SVMs

We now return to the **digit classification** problem introduced earlier (see Figure Figure 30).
Support Vector Machines are applied here to classify handwritten digits using a **One-Versus-One** scheme, where one binary classifier is trained for each pair of digits.
We compare SVMs using **linear**, **polynomial**, and **radial (Gaussian)** kernels. The tuning parameter C was selected by **10-fold cross-validation** to optimise test accuracy.

Training and test data:

- Training data $(x_1, y_1), \ldots, (x_n, y_n)$, $n = 7290$, are used for model fitting and parameter tuning.
  - Predictors x_i are greyscale pixel intensities of each image $\in \mathbb{R}^p$.
  - Outcomes y_i correspond to the true digit labels $\in \{0, 1, \ldots, 9\}$.
- Test data $(\hat{x}_1, \hat{y}_1), \ldots, (\hat{x}_m, \hat{y}_m)$, $m = 2006$, are used only for evaluating predictive performance.

Models compared:

- **linreg:** direct linear regression.
- **LDA1:** LDA based on the raw pixel values x_i.
- **LDA2:** LDA based on x_i and their squares x_i^2.
- **QDA:** Quadratic Discriminant Analysis with a separate covariance matrix per class.
- **log:** multinomial logistic regression.
- **log-ridge:** logistic regression with ridge penalty.
- **log-lasso:** logistic regression with lasso penalty.
- **RF:** random forest.
- **linear SVM:** support vector classifier with a linear kernel.
- **radial SVM:** support vector machine with an RBF kernel.

Training and test errors (%)

Method	Training	Test
linreg	7.6	13.1
LDA1	6.2	11.5
LDA2	3.9	10.2
QDA	1.8	13.5
log	0.01	11.1
log-ridge	4.1	9.0
log-lasso	2.7	8.8
RF	0.0	5.9
linear SVM	1.5	6.5
radial SVM	0.6	5.4

The results show that both **linear** and **radial** SVMs perform extremely well, with the **radial kernel** achieving the lowest test error (5.4%).
This demonstrates the flexibility of kernel-based SVMs in modelling complex, non-linear decision boundaries, even in high-dimensional spaces like image pixels.

As seen throughout this lecture, the **kernel trick** allows SVMs to implicitly map data into richer feature spaces without explicitly constructing new variables, resulting in powerful and computationally efficient classifiers.

7 Deep learning: Single hidden layer neural networks

7.1 Neural networks

In this part of the course we turn to **deep learning** and, in particular, to neural networks. We start from the most basic architecture, the **single hidden layer neural network**, and then move on to deeper models.

Neural networks have attracted a lot of attention in recent years and are often presented as almost magical tools that can solve any prediction problem. Our goal here is to **demystify** them and treat a (deep) neural network for what it is: a **highly flexible, non-linear prediction method**, comparable to methods you already know, such as random forests or support vector machines.

In our view, neural networks are simply mathematical models that are very good at prediction. They can be used for **regression**, when Y is quantitative, and for **classification**, when Y takes a finite number of classes. We will use the single hidden layer neural network as the basic building block to understand how these models are constructed and why they work well.

7.2 Single hidden layer neural network for regression

We now introduce the simplest neural network architecture for a quantitative response Y. As in previous regression methods, we start from p original predictors $X = (X_1, \dots, X_p)^\top$ and would like to build a flexible model for Y. The key idea of a single hidden layer neural network is to construct m new intermediate variables, called hidden units or hidden neurons Z_j, which play the role of automatically engineered features. Think of each hidden unit Z_j as a little calculator that does two operations in a row:

1. It computes a weighted sum of the inputs (plus a constant).
2. It squashes that number through a non-linear function.

Each hidden unit Z_j is obtained in two clear steps:

1. **Linear combination of the inputs.** We first compute a linear predictor

$$Z_j^{\text{lin}} = b_j^{(1)} + w_j^{(1)\top} X, \quad j = 1, \dots, m,$$

   which is a compact way of writing

$$Z_j^{\text{lin}} = b_j^{(1)} + w_{j1}^{(1)} X_1 + w_{j2}^{(1)} X_2 + \cdots + w_{jp}^{(1)} X_p.$$

where

- $w_j^{(1)} = (w_{j1}^{(1)}, \dots, w_{jp}^{(1)})^\top \in \mathbb{R}^p$ is the weight vector of the j-th hidden unit. in other words. In other words, $w_{j1}^{(1)}, \dots, w_{jp}^{(1)}$ are **weights** saying how important each feature is for this particular hidden unit and correspond to the arrows from the input nodes to a given hidden node in the network diagram.
- $b_j^{(1)} \in \mathbb{R}$ is its **bias** (an intercept term inside this inner linear model; it is unrelated to the "bias" from the bias–variance trade-off),
- Z_j^{lin} is therefore a standard linear combination of the original predictors.

Intuitively, this is like a grade computed as grade $= 5 + 0.5 \times$ exam $+ 0.3 \times$ project $+ 0.2 \times$ homework: multiply each input by a weight, add them up, add a constant. That is exactly what Z_j^{lin} does.

Geometrically, the equation $w_j^{(1)\top} X + b_j^{(1)} = 0$ defines a hyperplane (a line in 2D, a plane in 3D, etc.). The sign and size of Z_j^{lin} tell you on which side of that hyperplane you are and how far you are from it.

2. **Non-linear activation.** $Z_j = g(\cdot)$

   We then apply a non-linear activation function $g : \mathbb{R} \to \mathbb{R}$ (function that takes real numbers as input and produces real numbers as output) to this linear predictor,

$$Z_j = g(b_j^{(1)} + w_j^{(1)\top} X), \quad j = 1, \dots, m.$$

   This function g is exactly where the non-linearity of the model is introduced: if g were the identity, the hidden units would just be linear functions of X and the overall model would remain linear. By choosing a non-linear g (such as a sigmoid or ReLU, discussed later), we obtain transformed features $Z_1, \dots, Z_m$ that can capture complex relationships in the data. Importantly, the number m of hidden units does not need to equal p; it is a modelling choice that controls how rich this intermediate representation is.

   The final value Z_j is therefore a **non-linear feature** of the original inputs X. The network learns all the weights and biases so that these non-linear features become useful for predicting Y.

Intuitively:

- Step 1 (linear) measures **how strongly the pattern** the neuron is looking for is present in the inputs.
- Step 2 (activation) decides **how much the neuron fires** in response to that pattern.
  - With ReLU: if the linear combination is negative, the neuron stays silent (0); if it is positive, it passes the value on.
  - With sigmoid: the neuron responds smoothly between 0 and 1.

In summary, the first step of a single hidden layer neural network for regression takes the original predictors in the input layer and maps them to m hidden units via linear combinations plus a non-linear activation. Each hidden neuron collects signals from all input variables, weighs them according to its own weight vector, adds a bias, and then fires through g. These hidden units form a new set of features that will then be combined in a second step to model the response Y, as we will see next. The overall structure is sketched in Figure 24.

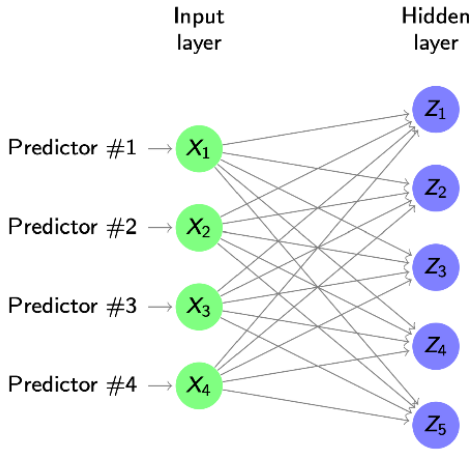

Figure 24: Single hidden layer neural network: inputs $X_1, \dots, X_p$ are linearly combined and passed through an activation function to form hidden units $Z_1, \dots, Z_m$.

7.3 Single hidden layer neural network for regression (output layer)

Once we have constructed the hidden units, we collect them in the vector $Z = (Z_1, \dots, Z_m)^\top$. In the regression setting we have a single quantitative output Y, so we now need to combine these m hidden neurons into a model for Y. We do this in exactly the same spirit as before: we take a linear combination of the hidden units and add an intercept, and optionally apply a further function h to this result, writing

$$\hat{f}(X) = h\big(b^{(2)} + w^{(2)\top} Z\big),$$

where $w^{(2)} = (w_1^{(2)}, \dots, w_m^{(2)})^\top \in \mathbb{R}^m$ are the second-layer weights and $b^{(2)} \in \mathbb{R}$ is the second-layer bias.

In regression we typically choose h to be the identity function, so the final model is simply a linear regression on the hidden units,

$$\hat{f}(X) = b^{(2)} + w^{(2)\top} Z.$$

Intuitively, each hidden neuron Z_j has already applied a non-linear transformation to the original inputs through the activation function g. The output neuron now just mixes these transformed features linearly. Because there is only one output neuron in regression, there is only one weight vector $w^{(2)}$ and one bias $b^{(2)}$, in contrast to the many weight vectors and biases in the hidden layer. The full network for regression can therefore be seen as two stacked linear models separated by the non-linear activation function g: first from X to Z, then from Z to Y.

Figure 25 visualises this whole process: the green nodes on the left represent the original predictors $X_1, \ldots, X_p$ (input layer), the blue nodes in the middle are the hidden units $Z_1, \ldots, Z_m$, and the red node on the right is the output Y. The arrows correspond to the weights in the two layers: first-layer weights from X to Z, and second-layer weights from Z to Y. Reading the diagram from left to right therefore mirrors exactly the two-step computation of the model $\hat{f}(X)$.

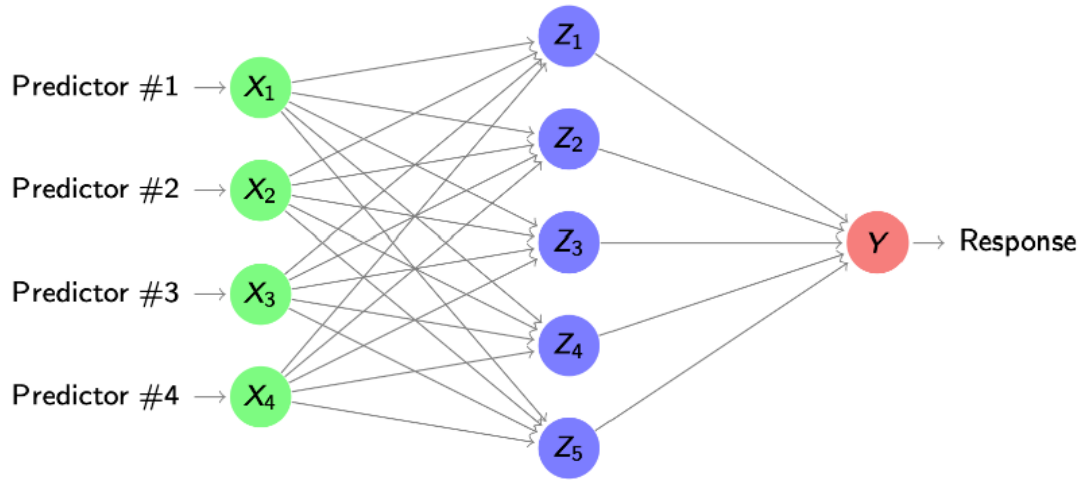

Figure 25: Single hidden layer neural network for regression: the hidden units $Z_1, \ldots, Z_m$ are combined linearly in the output neuron to produce the prediction for Y.

7.4 Biological motivation and notation

Artificial neural networks are loosely inspired by how neurons in the brain work, but only at a very high level. A biological neuron receives signals through its dendrites, combines them in the cell body, and, if the overall activation is large enough, sends an output impulse along its axon, as shown on the left of Figure 26.

The artificial neuron on the right of Figure 26 mirrors this structure in mathematical form. The inputs $x_1, x_2, \ldots$ correspond to incoming signals, each weighted by a connection strength w_i and summed together with a bias b. This sum is passed through an activation function $f(\cdot)$, which determines the output "firing" of the neuron. This loose analogy explains the biological terminology (neurons, activation, firing) used for what are, in the end, purely mathematical models.

7.5 Activation functions in the hidden layer

In matrix notation we can collect the m hidden units into the vector

$$Z = g(b^{(1)} + W^{(1)}X) \in \mathbb{R}^m,$$

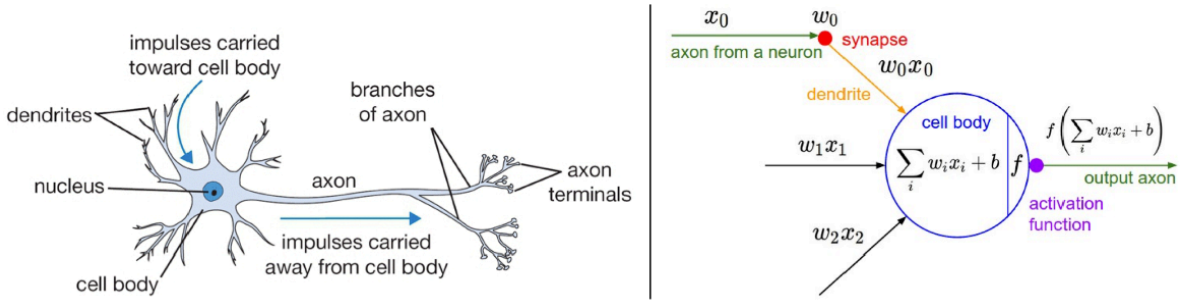

Figure 26: Biological neuron (left) and artificial neuron (right): loose analogy behind the terminology of neural networks.

where $W^{(1)} \in \mathbb{R}^{m \times p}$ contains the weights of the connections from the p inputs to the m hidden units, $b^{(1)} \in \mathbb{R}^m$ is the vector of biases, and g is applied componentwise to this vector.

The choice of activation function g is crucial. A traditional choice is the **sigmoid function**

$$\sigma(x) = \exp(x)/(1 + \exp(x)),$$

which smoothly squashes any real input to a value between 0 and 1, as shown in the left panel of Figure 27. In modern neural networks, the default is usually the **rectified linear unit (ReLU)**,

$$\text{ReLU}(x) = \max\{0, x\},$$

displayed in the right panel of Figure 27. ReLU leaves positive inputs unchanged and sets negative inputs to zero, which leads to simpler gradients and often faster, more stable training in deep networks.

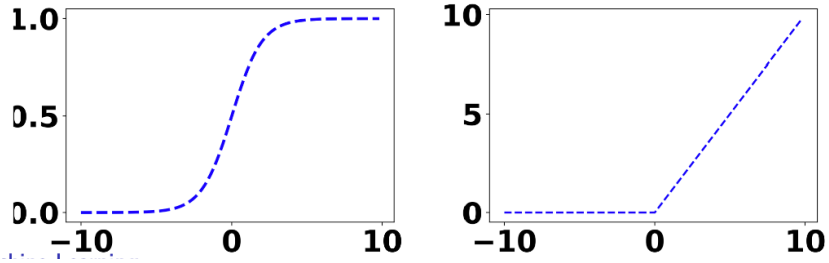

Figure 27: Sigmoid activation (left) and ReLU activation (right) as typical choices for the hidden layer non-linearity.

7.6 Single hidden layer neural network for regression

In regression, the output layer function h is typically just the identity $h(z) = z$, so the network prediction is a linear combination of the hidden units,

$$\hat{f}(X) = b^{(2)} + w^{(2)\top} Z.$$

If we now use ReLU in the hidden layer, the hidden vector is

$$Z = \max\{0,\, b^{(1)} + W^{(1)}X\},$$

where the max is applied componentwise. Plugging this expression for Z into the output layer gives a compact formula for the whole single hidden layer neural network,

$$\hat{f}(X) = b^{(2)} + w^{(2)\top} \max\{0,\, b^{(1)} + W^{(1)}X\}.$$

The diagram in Figure 25 summarises this computation visually: the inputs $X_1, \dots, X_p$ are first transformed by the ReLU hidden layer into $Z_1, \dots, Z_m$, and these hidden activations are then combined linearly in the single output neuron to produce the prediction for Y.

7.7 Single hidden layer neural network for classification

If the output is qualitative with q classes $Y \in \mathcal{G} = \{1, \dots, q\}$, we want the network to produce q class probabilities, one for each class. This is achieved by using q output neurons, each representing the probability of a particular class.

The hidden layer is unchanged and is still given in matrix form by

$$Z = g(b^{(1)} + W^{(1)}X),$$

where $Z \in \mathbb{R}^m$ is the vector of hidden activations. In the output layer we now use a weight matrix $W^{(2)} \in \mathbb{R}^{q \times m}$ and a bias vector $b^{(2)} \in \mathbb{R}^q$. For each class $i = 1, \dots, q$, the network first computes a score

$$s_i(X) = b_i^{(2)} + w_i^{(2)\top} Z,$$

where $w_i^{(2)}$ is the i-th row of $W^{(2)}$.

To turn these scores into probabilities we choose the output functions h_i to be the **softmax** components. Given q input scores $x_1, \dots, x_q$, the i-th softmax component is

$$h_i(x_1, \dots, x_q) = \text{softmax}(x_1, \dots, x_q)_i = \frac{\exp(x_i)}{\sum_{j=1}^{q} \exp(x_j)}, \quad i = 1, \dots, q.$$

This is exactly the same softmax function used in multinomial logistic regression. With ReLU in the hidden layer and softmax in the output layer, the single hidden layer neural network directly models the posterior probabilities

$$\hat{f}_i(X) = \Pr(Y = i \mid X) = \text{softmax}\big(b^{(2)} + W^{(2)} \max\{0,\, b^{(1)} + W^{(1)}X\}\big)_i, \quad i = 1, \dots, q.$$

For a new input $x_0 \in \mathbb{R}^p$ we then apply the Bayes classifier by predicting the class with the largest estimated probability,

$$\hat{G}(x_0) = \hat{y}_0 = \arg\max_{i=1,\dots,q} \hat{f}_i(x_0).$$

The architecture in Figure 28 visualises this: the green nodes represent the original predictors, the blue nodes are the shared hidden layer $Z_1, \ldots, Z_m$, and the red output nodes $Y_1, \ldots, Y_q$ compute the q class probabilities by applying softmax to different linear combinations of the same hidden activations.

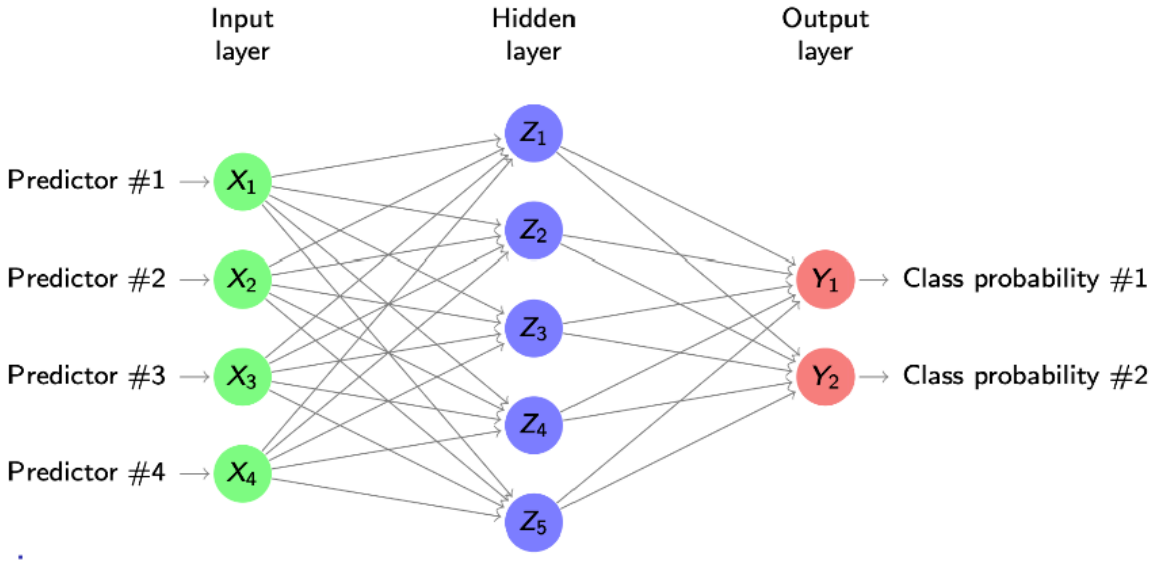

Figure 28: Single hidden layer neural network for classification: one output neuron per class, each modelling a class probability.

8 Deep Learning: Deep Neural Networks

8.1 Deep neural networks

A deep neural network is a neural network with **multiple hidden layers** between the input and the output layer. Instead of transforming the inputs X into a single hidden layer and then directly into the output, we now pass the information through a sequence of hidden layers, each building on the previous one.

Formally, we denote by $Z^{(1)}, \ldots, Z^{(M)}$ the hidden layers, where M is the number of hidden layers. The first hidden layer is constructed as

$$Z^{(1)} = g_1(b^{(1)} + W^{(1)}X),$$

where $W^{(1)}$ and $b^{(1)}$ are the weights and biases from the input to the first hidden layer, and g_1 is an activation function such as ReLU. Each subsequent hidden layer takes the previous hidden layer as input,

$$Z^{(2)} = g_2(b^{(2)} + W^{(2)}Z^{(1)}), \ldots, \; Z^{(M)} = g_M(b^{(M)} + W^{(M)}Z^{(M-1)}).$$

From the last hidden layer we still need to produce the outputs. For q outputs (for example q classes in classification), we use

$$\hat{f}_i(X) = h_i(b^{(M+1)} + W^{(M+1)} Z^{(M)}), \quad i = 1, \ldots, q,$$

where $W^{(M+1)}$ and $b^{(M+1)}$ are the weights and biases from the last hidden layer to the output layer, and h_i is the output function (identity for regression, softmax components for classification).

The network in Figure 29 visualises this construction: the green nodes are the input predictors, the blue nodes are the successive hidden layers $Z^{(1)}, Z^{(2)}, Z^{(3)}$, and the red output nodes Y_1, Y_2 are obtained by applying a final linear transformation and output functions to the last hidden layer.

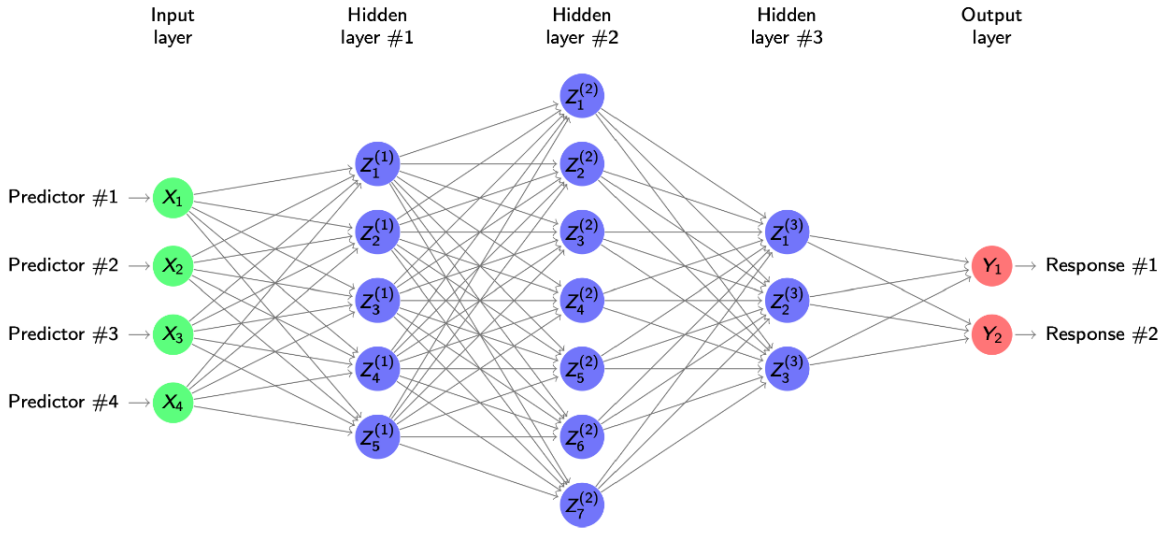

Figure 29: Deep neural network with several hidden layers between input and output: each hidden layer applies a linear transformation followed by a non-linear activation to the previous layer.

8.2 The architecture of deep neural networks

The **depth** of a neural network is the total number of layers, counting both hidden layers and the output layer (and sometimes also the input layer, depending on convention). For each layer, the **width** is the number of neurons in that layer. Together, depth and width largely determine the **architecture** of a deep neural network.

In practice there can be additional architectural choices, such as restricting which neurons are connected, or forcing some weights to be shared across different parts of the network (as in convolutional neural networks). A good architecture is usually found by combining prior experience with systematic comparison via cross-validation: we try several candidate depths and widths and keep the one that generalises best to new data.

The network shown in Figure 29 is an example of such an architecture: it has one input layer, three hidden layers of different widths, and an output layer with two neurons, each representing one response.

8.3 Comments on deep neural networks

In deep neural networks the default choice for all activation functions is usually the ReLU,

$$g_j(x) = \max\{0, x\}, \ j = 1, \dots, M,$$

because it is simple, cheap to evaluate, and works well in practice for many hidden layers.

For regression problems we typically have a single output ($q = 1$) and use the identity in the output layer,

$$h(z) = z,$$

so the network returns a real-valued prediction. For classification, the output functions h_i are the softmax components, which turn the final layer scores into class probabilities.

The hidden layers act as **feature extractors**: each layer transforms the representation from the previous one, gradually extracting higher-level patterns in the data. The first hidden layer, which is directly connected to the inputs, is particularly important for interpretation and visualisation of what features the network is learning.

Deep networks are **highly flexible**: with enough neurons they can approximate any continuous function on $\mathbb{R}^p$ to arbitrary accuracy (under mild assumptions). However, this flexibility comes at a cost. They contain **many parameters**, so fitting them is computationally expensive and there is a serious risk of overfitting. In practice, choosing and training a good architecture relies on experience plus careful use of validation and regularisation.

8.4 A simple example: digit classification

To see what these architectures look like in practice, consider a small digits data set where each image is represented by $p = 256$ grey-scale pixels. Each digit like those in Figure 30 is flattened into a vector $X \in \mathbb{R}^{256}$ that we feed into a deep neural network.

We take a **fully connected** network with $M = 3$ hidden layers, each containing 256 neurons, and an output layer with 10 neurons (one for each digit 0–9). The structure is shown in Figure 31: every neuron in one layer is connected to every neuron in the next one, so the information from each pixel can, in principle, influence every hidden unit and every output.

Even this "simple" architecture already has a large number of parameters. Each hidden layer has $(256+1) \times 256 = 257 \times 256$ weights and biases, and the output layer has $(256+1) \times 10 = 2570$ parameters. Altogether this is roughly

$$3 \times 257 \times 256 + 257 \times 10 \approx 200\,000$$

parameters to estimate. This example illustrates both the expressive power of deep networks and why training them is computationally demanding and prone to overfitting if not properly regularised.

Figure 30: Example images of handwritten digits used as inputs to the network.

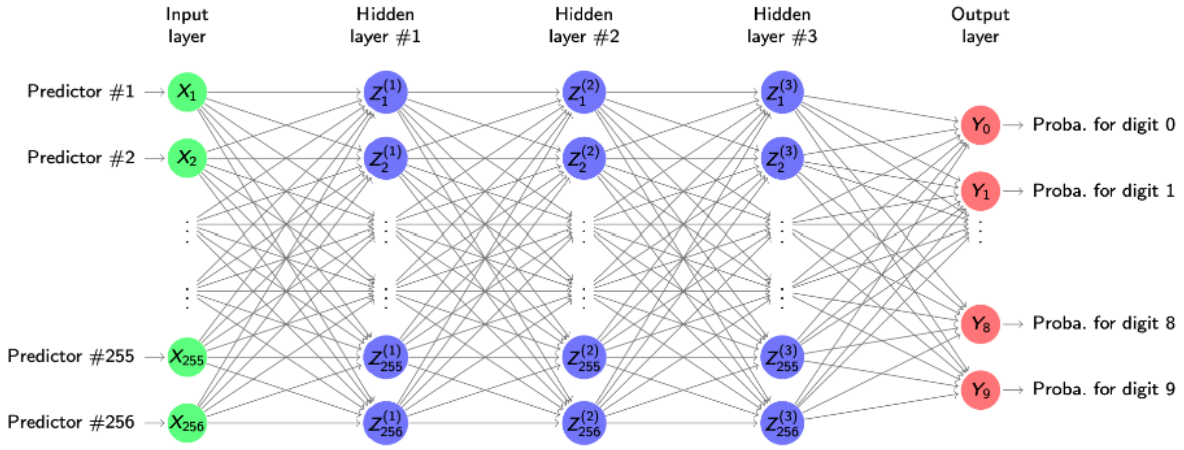

Figure 31: Fully connected deep neural network with three hidden layers of width 256 for digit classification.

9 Deep learning: Fitting neural networks and back-propagation

9.1 Loss functions for neural networks

Neural networks are non-linear parametric models. All weight matrices and bias vectors that transform one layer into the next are collected into a single parameter vector

$$\theta = (W^{(1)}, \dots, W^{(M+1)}, b^{(1)}, \dots, b^{(M+1)}).$$

For a fixed θ the network defines a predictive model; in regression we denote it by $\hat{f} = \hat{f}_\theta$, and in classification with q classes by $\hat{f}_1, \dots, \hat{f}_q$, where $\hat{f}_j(x)$ is the predicted probability of class j.

Given training data $(x_1, y_1), \dots, (x_n, y_n)$, we define a loss function $J(\theta)$ that measures how well the network predicts all responses on the training set. Changing θ changes the predictions $\hat{f}_\theta(x_i)$ (or $\hat{f}_j(x_i)$) and therefore changes the value of $J(\theta)$.

For regression, where $y_i \in \mathbb{R}$, we use the squared-error loss and define

$$J(\theta) = \sum_{i=1}^{n} \left(y_i - \hat{f}_\theta(x_i)\right)^2.$$

For classification with q classes, $y_i \in \{1, \dots, q\}$, we use the cross-entropy (deviance) loss

$$J(\theta) = -\sum_{i=1}^{n} \sum_{j=1}^{q} I\{y_i = j\} \log \hat{f}_j(x_i),$$

which is the negative log-likelihood of the multinomial model with class probabilities given by the network outputs $\hat{f}_1(x), \dots, \hat{f}_q(x)$.

9.2 Fitting of neural networks

Fitting a neural network means choosing the parameter vector θ (all weights and biases stacked into one long vector) so that the loss $J(\theta)$ is small on the training data. This is essentially an art and remains an active area of research. The loss $J(\theta)$ is a highly non-linear, non-convex function of very many parameters, so there is no closed-form solution: we cannot simply write down the optimal θ as we did for linear regression.

It is helpful to picture $J(\theta)$ as a complicated surface over a huge parameter space. In low dimension we might draw a bumpy landscape with many valleys and ridges; in reality the neural network lives in a very high-dimensional version of this picture. Because there are so many parameters, the true global (or even very deep local) minimum of $J(\theta)$ would typically correspond to overfitting the training data: the network can adapt too closely to idiosyncrasies and noise. In practice we are not aiming for that global minimum. Instead, we will later control overfitting with tools such as regularisation and early stopping and be satisfied with parameter values that give a good fit without memorising the data.

For now, the question is how we can improve an initial model using gradient descent. We start from some arbitrary initial parameter vector $\theta^{(0)}$ and move step by step down the loss surface.

Gradient descent for neural networks

1. Start with an arbitrary initial value $\theta^{(0)}$.
2. For iterations $i = 1, 2, \dots$:
    1. Compute the gradient (derivative) of the loss at the current parameters,

$$\nabla_\theta J(\theta^{(i-1)}) = \left.\frac{\partial J(\theta)}{\partial \theta}\right|_{\theta=\theta^{(i-1)}}.$$

2. Update the parameters with learning rate $\alpha > 0$,[3]

$$\theta^{(i)} = \theta^{(i-1)} - \alpha \nabla_\theta J(\theta^{(i-1)}).$$

3. Stop when a suitable stopping criterion is satisfied (for example, after a fixed number of iterations or when the loss stops improving).

Intuitively, each step looks at the current model, determines in which direction in parameter space $J(\theta)$ decreases fastest, and moves a small distance in that direction. The gradient descent algorithm itself is completely standard; what is special in the neural-network setting is how we obtain the gradient efficiently for such a complex model. In essence, to fit a neural network by gradient descent we "only" need the gradients $\nabla_\theta J(\theta)$, and these are supplied by the back-propagation algorithm.

9.3 Computing the gradient: back-propagation

In the sequel we focus on **regression**; the classification case is entirely analogous but uses the cross-entropy loss instead of the squared error. For regression with squared loss, the gradient of the total loss at some parameter vector θ can be written as a sum over the training samples,

$$\nabla_\theta J(\theta) = \sum_{i=1}^{n} \nabla_\theta \big(f_\theta(x_i) - y_i\big)^2.$$

This expression says that the overall direction in which we should move θ is obtained by adding up the contributions of each individual training example.

Because the gradient is a sum, we can look at a **single** training pair (x, y) and study its loss contribution

$$L(\theta) = \big(f_\theta(x) - y\big)^2.$$

If we understand how to differentiate this one-sample loss with respect to all parameters, then the full gradient is just the sum of these single-sample gradients. This is exactly the viewpoint that underlies back-propagation and, later, stochastic and mini-batch gradient descent.

Back-propagation is simply a way of computing $\nabla_\theta L(\theta)$ efficiently by exploiting the **chain rule** of calculus. The neural network takes x, pushes it through a sequence of linear transformations and non-linear activations, and finally produces the prediction $f_\theta(x)$. This whole computation is a long composition of simple functions. The chain rule tells us how the derivative of such a composition factors into the product of simpler derivatives.

Recall the chain rule for two functions f and g. If we define $h(x) = f(g(x))$, then

$$\left.\frac{\partial h(x)}{\partial x}\right|_{x=x_0} = \left.\frac{\partial f(z)}{\partial z}\right|_{z=g(x_0)} \cdot \left.\frac{\partial g(x)}{\partial x}\right|_{x=x_0}.$$

[3] α is the learning rate (step size) in gradient descent: it controls how far we move in the direction of the negative gradient at each update. A large α gives big, potentially unstable steps, while a small α yields slow but more stable progress.

In short-hand notation, if we set $z = g(x)$, this becomes

$$\frac{\partial h}{\partial x} = \frac{\partial h}{\partial z} \cdot \frac{\partial z}{\partial x}.$$

Interpretation:

1. See how h changes when z changes ($\partial h/\partial z$).
2. See how z changes when x changes ($\partial z/\partial x$).
3. Multiply them to get how h changes when x changes.

The key idea for back-propagation is to apply this rule repeatedly along the layers of the network, but **from the output back to the input**. We first differentiate the loss with respect to the final output $f_\theta(x)$, then propagate this derivative backwards through the last layer, then through the previous layer, and so on, each time multiplying by the relevant local derivative. In this way we reuse intermediate quantities and avoid recomputing the same derivatives many times. This is what makes back-propagation an efficient method for obtaining the gradients needed in gradient descent for neural networks.

In other words, back-propagation goes from the output layer to the input layer, using the chain rule to compute how the loss changes with respect to each layer's activations and weights, reusing intermediate results so we don't recompute the same derivatives again and again. This makes gradient computation efficient.

9.4 Back-propagation: the forward pass

The forward pass takes a fixed parameter vector θ and a single training pair (x, y), sends x from the input layer through all hidden layers to the output, computes the prediction $\hat{y}$ and its loss against y, and stores all intermediate activations. These stored activations are exactly what back-propagation will use later to compute gradients efficiently.

To compute the gradient for a given parameter vector θ it is convenient to look at one training pair (x, y) at a time. We fix a single input vector $x = (x_1, \dots, x_p)$ and its target y, together with the current weights and biases collected in θ. For this fixed θ we first run a **forward pass** through the network in order to compute the activation value of every hidden neuron and the final prediction.

Starting from the input layer, the activations of the hidden units are computed exactly as in the definition of the network. For a single hidden layer with activation function g this means

$$z = g\big(b^{(1)} + W^{(1)}x\big),$$

where $W^{(1)}$ and $b^{(1)}$ are the weights and biases of the first layer and z collects the hidden activations. The output neuron then combines these hidden activations linearly to form the prediction

$$\hat{y} = f_\theta(x) = b^{(2)} + w^{(2)\top} z.$$

Conceptually, the forward pass in Figure 32 takes the fixed input x (green nodes on the left), pushes it through the hidden layer (blue nodes z_1, z_2), and produces the output $\hat{y}$ (red node). We then compare $\hat{y}$ with the true target y (green node on the right) and evaluate the loss

$$L(\theta) = (\hat{y} - y)^2.$$

In deeper networks the same idea applies: we repeatedly apply "linear transformation + non-linearity" from layer to layer until we reach the output. All these intermediate quantities (the activations in each layer and the prediction) will then be reused in the backward pass to compute the gradients efficiently via the chain rule.

Summary of the forward pass for one sample (x, y) (as in Figure 32):

- Fix the input vector $x = (x_1, \dots, x_p)$, the target y and the current parameters θ.
- Compute the hidden activations by propagating x through the first layer: $z = g(b^{(1)} + W^{(1)}x)$.
- Compute the network prediction at the output layer: $\hat{y} = b^{(2)} + w^{(2)\top} z$.
- Evaluate the loss contribution of this sample: $L(\theta) = (\hat{y} - y)^2$.

These forward computations provide all the activations that back-propagation will differentiate with respect to the parameters in the backward pass.

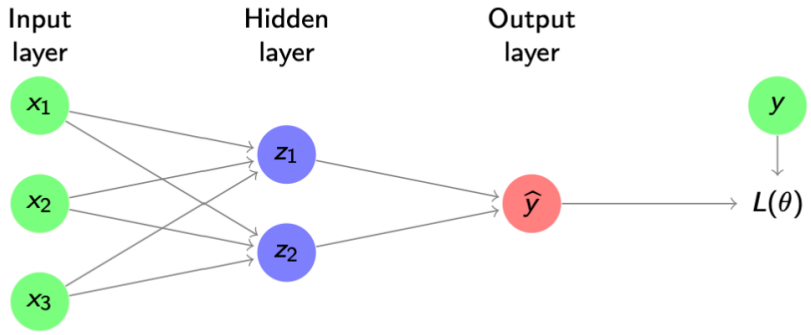

Figure 32: Forward pass for one training example (x, y): the input is propagated through the hidden layer to the output and the loss $L(\theta)$ is computed.

As a side product we also get the activations of the hidden layers.

9.5 Back-propagation

From the forward pass we know that, for a single hidden layer,

$$z = g(b^{(1)} + W^{(1)}x), \qquad \hat{y} = f_\theta(x) = b^{(2)} + w^{(2)\top} z, \qquad L(\theta) = (\hat{y} - y)^2.$$

Here z contains the hidden activations, $\hat{y}$ is the prediction for the fixed input x, and $L(\theta)$ is the squared-error loss for this one training pair, exactly as in Figure 32.

To obtain the **gradient**, we differentiate $L(\theta)$ with respect to **all parameters** in the network, starting from the output layer and moving backwards.

9.5.1 Output-layer weights

Suppose the output-layer weight vector is $w^{(2)} = (w_1^{(2)}, w_2^{(2)})$ as in Figure 32. We first look at $\partial L / \partial w_1^{(2)}$, i.e. how much the loss changes if we slightly change the connection from hidden unit z_1 to the output.

Back-propagation applies the **chain rule**:

$$\frac{\partial L}{\partial w_1^{(2)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_1^{(2)}}.$$

For the squared loss $L(\theta) = (\hat{y} - y)^2$ we have

$$\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y),$$

which is just (twice) the prediction error. Since the output is linear in the hidden activations,

$$\hat{y} = b^{(2)} + w^{(2)\top} z = b^{(2)} + w_1^{(2)} z_1 + w_2^{(2)} z_2,$$

we obtain

$$\frac{\partial \hat{y}}{\partial w_1^{(2)}} = z_1.$$

Putting these pieces together gives

$$\frac{\partial L}{\partial w_1^{(2)}} = 2(\hat{y} - y)\, z_1.$$

By the same argument,

$$\frac{\partial L}{\partial w_2^{(2)}} = 2(\hat{y} - y)\, z_2,$$

and in general

$$\frac{\partial L}{\partial w_i^{(2)}} = 2(\hat{y} - y)\, z_i.$$

Thus each gradient component at the output layer is the product of an **error term** $(\hat{y} - y)$ at the red node and the corresponding **hidden activation** z_i at the blue node in Figure 32.

9.5.2 First-layer weights

We now move one step further back and compute derivatives with respect to the **first-layer weights**. Focus first on a single weight $w_{11}^{(1)}$, which connects input x_1 to hidden unit z_1 in Figure 32. Along this path the loss depends on $w_{11}^{(1)}$ via

$$w_{11}^{(1)} \;\longrightarrow\; z_1 \;\longrightarrow\; \hat{y} \;\longrightarrow\; L,$$

so the chain rule gives

$$\frac{\partial L}{\partial w_{11}^{(1)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_{11}^{(1)}}.$$

We already know $\partial L/\partial \hat{y} = 2(\hat{y} - y)$. The second factor is the sensitivity of the output to the hidden unit:

$$\frac{\partial \hat{y}}{\partial z_1} = w_1^{(2)},$$

because $\hat{y} = b^{(2)} + w_1^{(2)} z_1 + w_2^{(2)} z_2$. For the third factor we use the definition of the hidden activation,

$$z_1 = g(b_1^{(1)} + w_1^{(1)\top} x),$$

where $w_1^{(1)}$ is the row of $W^{(1)}$ corresponding to hidden unit z_1. Differentiating with respect to $w_{11}^{(1)}$ yields

$$\frac{\partial z_1}{\partial w_{11}^{(1)}} = g'\big(b_1^{(1)} + w_1^{(1)\top} x\big)\, x_1.$$

Combining the three factors we obtain the explicit formula from the slide:

$$\frac{\partial L}{\partial w_{11}^{(1)}} = 2(\hat{y} - y) \cdot w_1^{(2)} \cdot g'\big(b_1^{(1)} + w_1^{(1)\top} x\big)\, x_1.$$

Exactly the same logic applies to any first-layer weight $w_{ij}^{(1)}$, which connects input x_j to hidden unit z_i. Along the path

$$w_{ij}^{(1)} \;\longrightarrow\; z_i \;\longrightarrow\; \hat{y} \;\longrightarrow\; L$$

we obtain

$$\frac{\partial L}{\partial w_{ij}^{(1)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_i} \cdot \frac{\partial z_i}{\partial w_{ij}^{(1)}} = 2(\hat{y} - y) \cdot w_i^{(2)} \cdot g'\big(b_i^{(1)} + w_i^{(1)\top} x\big)\, x_j.$$

This formula shows the typical **back-propagation structure** for a hidden-layer weight:

- an error term $2(\hat{y} - y)$ coming from the output layer,
- multiplied by the weight $w_i^{(2)}$ that connects hidden unit z_i to the output,

- multiplied by the derivative of the activation g' evaluated at the hidden pre-activation $b_i^{(1)} + w_i^{(1)\top} x$,
- multiplied by the input x_j to that hidden unit.

In words, the error at the output node is propagated backwards through the network, scaled at each step by the relevant weights and activation derivatives. This is precisely why the algorithm is called **back-propagation** and why it efficiently provides gradients for all parameters using only local computations along the graph in Figure 32.

9.5.3 Error terms and compact notation

The formulas we obtained for the weight derivatives can be written more compactly by introducing **error terms** (often called *deltas*).

For the output-layer weights we had

$$\frac{\partial L}{\partial w_i^{(2)}} = 2(\hat{y} - y)\, z_i.$$

We now define a single scalar

$$\delta^{(2)} = 2(\hat{y} - y),$$

which is the error at the output neuron for the current parameters θ. Then

$$\frac{\partial L}{\partial w_i^{(2)}} = \delta^{(2)} z_i.$$

For the first-layer weights we found

$$\frac{\partial L}{\partial w_{ij}^{(1)}} = 2(\hat{y} - y) \cdot w_i^{(2)} \cdot g'(b_i^{(1)} + w_i^{(1)\top} x)\, x_j.$$

We now define, for each hidden unit i,

$$\delta_i^{(1)} = 2(\hat{y} - y) \cdot w_i^{(2)} \cdot g'(b_i^{(1)} + w_i^{(1)\top} x),$$

so that

$$\frac{\partial L}{\partial w_{ij}^{(1)}} = \delta_i^{(1)} x_j.$$

The quantities $\delta^{(2)}$ and $\delta_i^{(1)}$ are the **errors** associated with the output neuron and the i-th hidden neuron, respectively. They satisfy the simple relationship

$$\delta_i^{(1)} = \delta^{(2)} \cdot w_i^{(2)} \cdot g'(b_i^{(1)} + w_i^{(1)\top} x),$$

which is exactly the expression on the slide: the error at a hidden node is the error at the next layer, multiplied by the connecting weight and by the derivative of the activation at that node.

For the ReLU activation $g(x) = \max\{0, x\}$ we have

$$g'(x) = 1\{x > 0\},$$

so the hidden error $\delta_i^{(1)}$ is either zero (if the neuron is inactive) or proportional to $\delta^{(2)} w_i^{(2)}$ (if the neuron is active).

9.5.4 Two-pass back-propagation algorithm

With this delta notation, back-propagation is implemented as a **two-pass algorithm**:

1. **Forward pass.**
   Compute all hidden activations z_i and the prediction $\hat{y}$ for the current θ, then evaluate the loss $L(\theta) = (\hat{y} - y)^2$.
2. **Backward pass.**
   - Compute the output error $\delta^{(2)} = 2(\hat{y} - y)$.
   - Use the recursion

     $$\delta_i^{(1)} = \delta^{(2)} \cdot w_i^{(2)} \cdot g'(b_i^{(1)} + w_i^{(1)\top} x)$$

     to obtain the hidden errors $\delta_i^{(1)}$.
   - Finally compute the gradients

     $$\frac{\partial L}{\partial w_i^{(2)}} = \delta^{(2)} z_i, \qquad \frac{\partial L}{\partial w_{ij}^{(1)}} = \delta_i^{(1)} x_j.$$

The equations for the derivatives with respect to the biases are analogous: each bias gradient is just the corresponding delta (for the neuron it belongs to). In deeper networks, the same pattern repeats layer by layer: errors are first computed at the output, then **back-propagated** through the network using formulas of the form above, and the gradients with respect to all weights and biases follow from these deltas.

9.6 Advantages of back-propagation

The delta notation extends directly from a single hidden layer to **deep networks** with many hidden layers. In a network with hidden layers $Z^{(1)}, \ldots, Z^{(M)}$ and output $\hat{y}$, we have an error vector $\delta^{(m)}$ at each hidden layer $m = 1, \ldots, M$ and an output error $\delta^{(M+1)}$ at the last layer. Back-propagation

computes these errors starting from the output and then propagating them backwards through the layers, exactly as we did for the single hidden layer.

As illustrated in Figure 33, each blue hidden layer has its own error vector $\delta^{(m)}$. The recursion has always the same structure: the error at layer m is obtained from the error at layer $m+1$ by multiplying with the corresponding weight matrix and the derivative of the activation function at that layer. Once all $\delta^{(m)}$ have been computed, the gradients with respect to all weights and biases follow from simple local formulas such as

$$\frac{\partial L}{\partial w_{ij}^{(m)}} = \delta_i^{(m)} a_j^{(m-1)},$$

where $a_j^{(m-1)}$ is the activation coming into layer m (for $m = 1$ this is just the input x_j).

Back-propagation has several important advantages:

- It is **very efficient**: the same forward activations and backward error terms $\delta^{(m)}$ are reused many times when computing the gradients, so the cost of obtaining all partial derivatives is only a small constant factor larger than a single forward pass.
- Only **local derivatives** are needed at each node: we never differentiate the full composition explicitly, but only the activation function at that node and the local linear transformation. This locality makes the algorithm easy to implement for a wide range of architectures.
- The computations at different neurons and different training examples are **embarrassingly parallel**, so back-propagation can be implemented very efficiently on GPUs and other parallel hardware.

In summary, back-propagation scales well to deep neural networks precisely because it organises gradient computation into local, reusable pieces: forward activations $a^{(m)}$ and backward errors $\delta^{(m)}$ that flow through the layered structure in Figure 33.

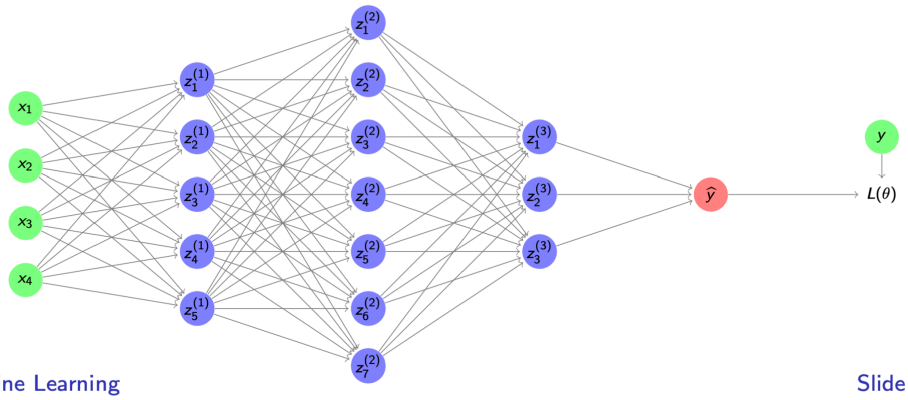

Figure 33: Back-propagation in a deep network: each hidden layer has its own error vector $\delta^{(m)}$, which is propagated backwards from the output to the input layer.

9.7 Batch optimisation

Recall from the gradient descent section that the loss for regression is

$$J(\theta) = \sum_{i=1}^{n} \left(y_i - \hat{f}_\theta(x_i)\right)^2,$$

so its gradient is the sum of the per-sample gradients

$$\nabla_\theta J(\theta) = \sum_{i=1}^{n} \nabla_\theta \left(y_i - \hat{f}_\theta(x_i)\right)^2.$$

Back-propagation gives us exactly these single-sample gradients: for each training pair (x_i, y_i) we can run a forward pass, compute the loss, and then a backward pass to obtain $\nabla_\theta \left(y_i - \hat{f}_\theta(x_i)\right)^2$.

To improve the error function $J(\theta)$ we now have two options; the first one is **batch optimisation** (the classical setting in the script):

- For each update step we use the **whole training set** $(x_1, y_1), \ldots, (x_n, y_n)$.
- We compute all per-sample gradients via back-propagation and sum them to obtain the exact gradient $\nabla_\theta J(\theta)$ as in the formula above.
- We then take a gradient descent step

  $$\theta \leftarrow \theta - \alpha \, \nabla_\theta J(\theta),$$

  where $\alpha > 0$ is the learning rate.

This procedure is simply **standard gradient descent** applied to neural networks: every step moves the parameters in the direction of the *true* gradient based on all observations. The advantage is that each update uses the most accurate possible information about how $J(\theta)$ changes. The drawback, emphasised in the video, is computational: when n is large, each step requires running back-propagation over the full dataset, which can be very slow. This motivates mini-batch and stochastic variants discussed on the following slides.

9.8 Stochastic gradient descent

For regression the loss is

$$J(\theta) = \sum_{i=1}^{n} \left(y_i - \hat{f}_\theta(x_i)\right)^2,$$

so the exact gradient needed for gradient descent is

$$\nabla_\theta J(\theta) = \sum_{i=1}^{n} \nabla_\theta \left(y_i - \hat{f}_\theta(x_i)\right)^2.$$

In batch optimisation we use **all** n training samples to compute this gradient before each update. As highlighted in the script, when n is very large (millions or billions of examples), a single full gradient step already becomes prohibitively slow.

To make learning feasible we approximate this gradient using much smaller **mini-batches**.

9.8.1 Mini-batch optimisation and stochastic gradient descent

- Randomly shuffle the training data and split it into $\frac{n}{m}$ **mini-batches** of size m, where $m \ll n$ (typically from 1 to a few hundred).
- In each step, stochastic gradient descent picks one mini-batch $(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})$ and uses only these m examples to approximate the full gradient:

$$\nabla_\theta J(\theta) \approx \sum_{i=1}^{m} \nabla_\theta \big(y^{(i)} - \hat{f}_\theta(x^{(i)})\big)^2.$$

- We then update the parameters using this **stochastic** gradient:

$$\theta \leftarrow \theta - \alpha \sum_{i=1}^{m} \nabla_\theta \big(y^{(i)} - \hat{f}_\theta(x^{(i)})\big)^2.$$

Each mini-batch therefore gives a **noisy estimate** of the true gradient. A single step is not optimal, but it is very fast to compute, and by taking many such steps we still move, on average, in a good descent direction.

9.8.2 Intuition and comparison with batch optimisation

- Mini-batch optimisation takes **many small, cheap steps** that are only approximately optimal but can be computed very quickly.
- Batch optimisation takes **few very accurate steps**, but each requires a pass over all n samples and is therefore much slower and often infeasible for large datasets.
- In modern applications, many training samples are **similar or redundant** (for example, similar images). Using a mini-batch discards little information while dramatically reducing computation.
- A **training epoch** is one full sweep over the training set: each observation has appeared in at least one mini-batch update. In practice we run several epochs, letting these noisy updates gradually decrease $J(\theta)$ as the parameters θ are refined.

10 Deep learning: Regularization for neural networks

10.1 Parameter initialization

Fitting deep neural networks is a difficult task and, as the professor stresses, "an art and an area of research in itself". The loss $J(\theta)$ is highly non-convex, with many local minima and no closed-form solution for the optimal parameters, so gradient descent will typically not find **the** global minimum, but just slide down to some local minimum near the starting point $\theta^{(0)}$.

Because of this, the way we **initialise** the parameters already acts as a first form of regularisation:

- It is usually advisable to **standardise the inputs** so that each predictor has mean zero and standard deviation one. Then all coordinates of $x = (x_1, ..., x_p)$ are on a comparable scale and are "treated equally", which matters especially once we add explicit regularisation.
- Since $J(\theta)$ is non-convex, the **starting values** $\theta^{(0)}$ can have a strong influence on the final solution we reach with gradient descent. Different random initialisations may lead to different local minima, some of which generalise better than others.
- A general recommendation in the script is to initialise all weights as **small random numbers**, for example drawn from a normal or uniform distribution centred at zero.
  - If the initial weights are **too large**, then in the forward pass repeated linear combinations make activations **explode**: values become huge, the initial predictions are extremely poor, and we may never reach a "reasonable" region of parameter space.
  - If the initial weights are **too small**, we **lose signal** during forward and back-propagation: activations and gradients become tiny, so essentially nothing moves and the network does not learn.

Good initialisation therefore aims for a **compromise**: small random weights that are large enough to keep some signal flowing through the layers, but small enough to avoid exploding activations and gradients. Modern schemes (e.g. Xavier, He) formalise this idea by choosing the variance of the initial weights as a function of the layer width, but the basic intuition is exactly the one explained in the video.

10.2 Regularisation

Deep neural networks have **a huge number of parameters**. If we simply ran gradient descent all the way to (a) minimum of the training loss $J(\theta)$, the training error would become extremely small, but we would usually **overfit**: the model would adapt to noise and idiosyncrasies of the training set, and its **prediction error** on new data would be poor.

Regularisation explicitly **prevents overfitting**: we sacrifice some training fit in order to improve the **generalisation error** (test error). The script presents several complementary tools:

10.2.1 Parameter penalties

As in ridge and lasso regression, we can add a **penalty term** to the loss and minimise the **penalised objective**

$$J_{\text{reg}}(\theta) = J(\theta) + \alpha\Omega(\theta),$$

where $\alpha > 0$ is a tuning parameter and $\Omega(\theta)$ measures the overall size of the parameters.

A common choice is an ℓ_2-type penalty on the weights:

$$\Omega(\theta) = \sum_{m,i,j} \left(w_{ij}^{(m)}\right)^2,$$

i.e. we sum the squared values of all weights $w_{ij}^{(m)}$ over all layers m and connections (i, j); often biases are excluded or penalised more weakly. This **shrinks the weights towards zero** and makes the network less "wiggly", in exactly the same spirit as ridge regression. One can also use an ℓ_1 penalty to encourage sparsity, but in neural nets we typically care less about exact zero coefficients than we did in lasso, because individual weights have no simple interpretation.

Interpretation:

- $J(\theta)$ wants to **fit the training data** as well as possible.
- $\Omega(\theta)$ wants the **weights to stay small**, keeping the model simple.
- α balances these two forces: large α means strong regularisation (simpler networks, higher bias, lower variance); small α means weak regularisation (very flexible networks, lower bias, higher variance).

As in other regularised models, α is typically chosen by **validation or cross-validation**, looking at the behaviour of the **validation error**, not the training error.

10.3 Dropout

Dropout is presented in the script as a **completely different** type of regularisation: instead of adding a penalty, we change the **network structure at every gradient step**.

Before each forward pass (and the corresponding back-propagation), **every non-output neuron is independently dropped with some probability** $q \in [0, 1]$:

- for hidden units, a typical choice is $q \approx 0.5$ (so each hidden neuron is kept with probability 0.5);
- for input units, a smaller probability is used, e.g. $q \approx 0.2$ (each input feature is kept with probability 0.8).

If a unit is dropped, its activation is set to zero and none of its outgoing weights contribute in that update. In that gradient step we pretend that the network **never had** those grey units. We compute the gradient only on the **reduced network**, and only the weights belonging to currently active connections are updated.

In Figure 34 you see this visually: the grey input and hidden units are dropped for the current step, while the blue units form the "thinned" network that actually propagates the signal from the inputs to the outputs.

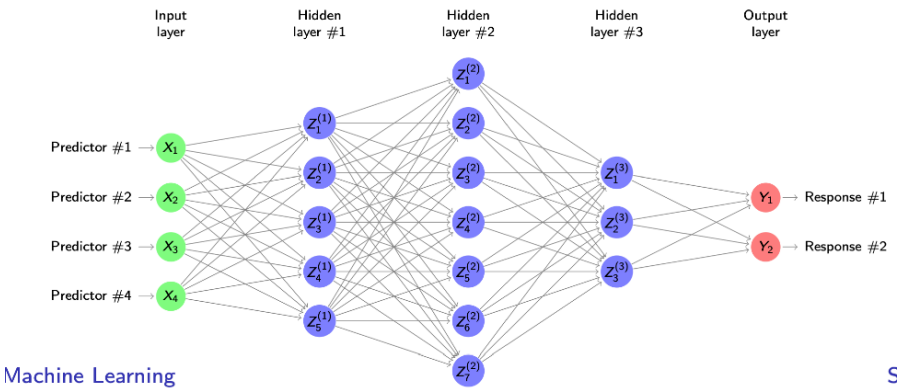

Figure 34: Dropout in one gradient step: grey units are dropped and only the blue "thinned network" is used for forward and backward passes.

This procedure **robustifies the fitting**:

- Each weight that is updated must learn to work **independently of particular neighbours**, because some of those neighbours might vanish in the next iteration.
- The network cannot rely on a single fragile pathway; useful information must be distributed across many different paths through the network.

The script then makes a clear analogy to **bagging / random forests**: if we look at the input layer, dropout means some predictors (say X_3) are simply not used in that step, just like in random forests we exclude some variables in individual trees. Over many gradient steps, we are effectively training **many slightly different networks**, each seeing a different subset of units.

In simple cases one can interpret the final model as a kind of **ensemble average** over all these thinned networks: we combine the fitted weights and the dropout probabilities, just as bagging combines predictions from bootstrap samples. This ensemble view explains why dropout leads to a more **robust final predictor**. Figure 35 illustrates this "ensemble of thinned networks" perspective.

At **test time** we use the **full network without dropout**, but rescale activations or weights so that the expected activation of each unit matches what it saw during training. Operationally, dropout behaves like a cheap built-in ensemble method that regularises deep networks and improves generalisation.

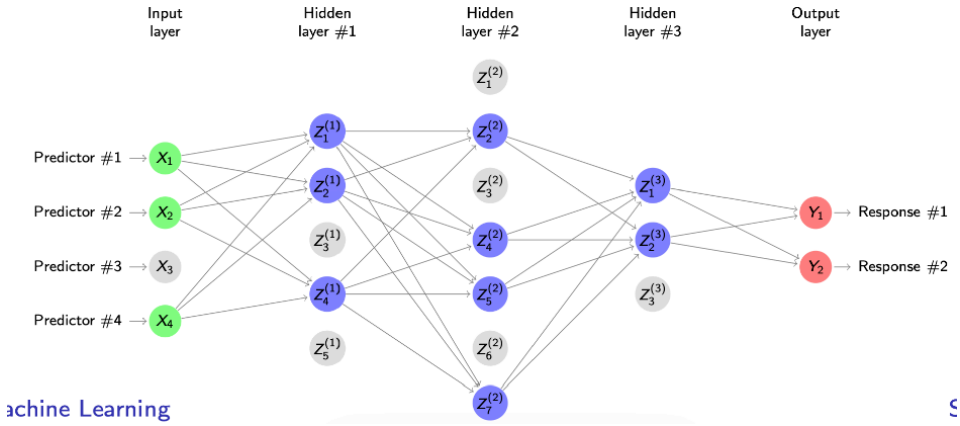

Figure 35: Dropout as an implicit ensemble: across many training steps, different thinned networks are fitted, and the final predictor behaves like an average over these models.

11 Deep learning: Convolutional Neural Networks

11.1 Image recognition: CIFAR-10

To illustrate convolutional networks, we will work with the CIFAR-10 data set, a standard benchmark for image recognition. It consists of 60 000 colour images of size (32×32) pixels, each belonging to one of 10 object classes:

$$\mathcal{G} = \{\text{airplane}, \text{automobile}, \text{bird}, \text{cat}, \text{deer}, \text{dog}, \text{frog}, \text{horse}, \text{ship}, \text{truck}\}.$$

Each image has **three colour channels** (red, green, blue). At every pixel we therefore store three intensity values, one per channel. If we ignore the spatial structure and simply stack all pixel values into a single vector, the predictor dimension would be

$$p = 32 \times 32 \times 3 = 1024.$$

So each tiny CIFAR-10 image is already a 1024-dimensional point in input space. Many images are also quite ambiguous or noisy, making them challenging to classify even for humans. Some examples are shown in Figure 36. Yet modern convolutional neural networks, which explicitly exploit the 2D image structure and local patterns (edges, textures, shapes), reach test accuracies above 95 %. CIFAR-10 is therefore a convenient running example: small enough to train models quickly, but rich enough to show why specialised architectures for images are needed.

11.2 A dense neural network

For the CIFAR-10 data set, each image is represented by $(p = 1024)$ predictors (32 × 32 pixels × 3 colour channels).

Figure 36: Examples of CIFAR-10 images

A **dense** (fully connected) neural network connects every neuron in one layer to every neuron in the next, each connection having its own weight. If we build a dense network for CIFAR-10 with hidden layers of size 1024, then each dense layer contains roughly

$$1024 \times 1024 \approx 10^6$$

weights, plus biases. So each layer has about one million parameters, and the full network has several million.

This creates two main problems:

- **Computational cost:** backpropagation has to update millions of parameters at every training step.
- **Overfitting:** with only 60 000 training images, the model has far more parameters than observations and can easily memorise the training set instead of learning patterns that generalise.

As illustrated in Figure 37, this architecture also ignores the spatial structure of images: the pixels are simply stacked into a long vector $X_1, \ldots, X_{1024}$, and the network learns separate weights for each connection. This motivates convolutional neural networks, which drastically reduce the number of parameters by sharing weights and by explicitly exploiting locality in images.

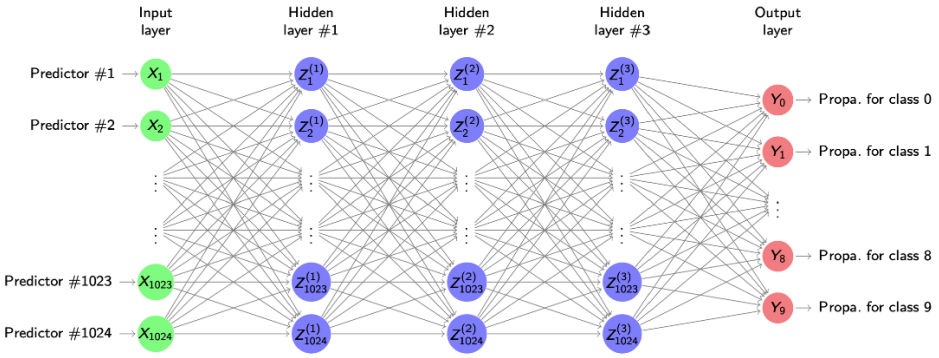

Figure 37: Dense fully connected neural network for CIFAR-10 classification

11.3 Spatial structures in image data

So far we have treated images as long predictor vectors $X_1, \ldots, X_{1024}$ and used generic regularisation (ridge, dropout, etc.) to fight overfitting. For image data, however, we have extra **domain knowledge**

we can exploit: there is a natural **spatial structure** in the predictors. Neighbouring pixels in an image tend to look similar, while pixels that are far apart are usually less related.

A very effective way to regularise a neural network is therefore to **decrease the number of parameters in its architecture**, using this spatial structure. Instead of learning completely separate weights for every pixel–neuron pair, we can **share parameters** across locations that "play the same role" in different parts of the image (e.g. all local 3×3 patches). This reduces model complexity and builds in the prior belief that local patterns (edges, corners, small shapes) repeat across the image.

Figure 38: CIFAR-10 car example illustrating spatial structure in neighbouring pixels

In the CIFAR-10 car image shown in Figure 38, for example, neighbouring pixels along the car roof or the road are very similar, whereas a pixel on the sky and a pixel on a tyre are quite different. Convolutional neural networks make this intuition explicit in their architecture: they use local filters that look at small neighbourhoods of pixels and **reuse the same filter weights across the whole image**, achieving parameter sharing and exploiting spatial structure at the same time.

11.4 Convolutional neural networks

Convolutional neural networks (CNNs) are **deep neural networks** with many hidden layers, designed specifically to exploit the spatial structure of image data.

The key idea is that hidden layers no longer all have to be of the same **dense/fully connected** type. So far we only know dense layers, where every neuron is connected to all neurons of the previous layer with its own weight. CNNs introduce other layer types in which:

- different operations are applied, and/or
- **parameter sharing** is used instead of separate weights for every connection.

Among these, the most important are **convolutional layers**, whose role is to extract **local image features** (such as edges or small patterns). Conceptually, a convolutional layer is built from three stages:

a) **Convolution stage:** linear filtering with small local kernels to produce feature maps
b) **Activation stage:** a non-linearity (typically ReLU) applied pointwise to these feature maps;
c) **Pooling stage:** a downsampling step (e.g. max pooling) that aggregates nearby activations.

In the following slides we will study each of these stages in detail and see how stacking such layers allows CNNs to learn increasingly complex visual features.

11.5 The convolutional stage

The first stage of a convolutional layer is the **convolution operation**. It takes a small local patch of the image, called the **receptive field** (for example a 3×3 block of pixels), and combines its values linearly into a single score. The weights of this linear combination form a **filter** (or kernel).

In the illustration Figure 39, the left grid is the input image I, the middle grid is a 3×3 filter K, and the right grid is the **convolved image** $I * K$:

- We place the filter on top of a 3×3 receptive field in I (red square).
- We multiply each entry of the receptive field by the corresponding entry of K and sum all products.
- This sum is written into the corresponding position of the output grid $I * K$ (green cell).

Formally, for a receptive field of the same size as K, the output at location (i, j) is

$$(I * K)_{ij} = \sum_{u,v} I_{i+u,\, j+v}\, K_{u,v}.$$

So convolution is just a dot product between the filter and the local image patch.

We then **slide** the receptive field across the image: first to the right, then down, repeating the same computation at each location. This produces a whole grid of scores, often called a **feature map**. Intuitively, if the pattern in the receptive field looks similar to the filter (for example, both look like a small cross or an edge), the score will be large; if they are very different, the score will be small. Different filters therefore detect different local features.

The crucial point is that the **same filter weights are reused** at every spatial position. The convolutional stage of a layer has only a small number of parameters (the entries of K plus a bias), shared across all locations. Compared with a dense layer, this dramatically reduces the number of parameters while still exploiting the local structure of images.

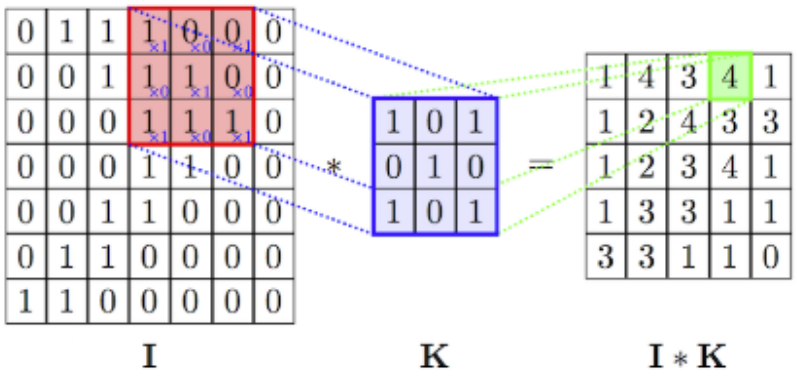

Figure 39: Illustration of a 3×3 convolution: a receptive field in the input image I is combined with a filter K to produce one entry of the convolved image I * K.

11.6 The convolution stage: hyperparameters

Convolutional layers come with several important **hyperparameters** that control how the filter is applied and how large the output feature map will be.

- **Filter width and height.**
  The first choice is the spatial size of the filter (for example 3×3 or 5×5). Larger filters see a bigger neighbourhood but also increase the number of parameters and computations.
- **Depth (number of filters).**
  A single filter produces one output feature map. In practice we use many different filters in parallel, so the **depth** of the layer is the number of feature maps produced. For image input with three colour channels, a convolutional layer with, say, 32 filters will output 32 feature maps, each responding to a different type of local pattern.
- **Stride.**
  The **stride** specifies by how many pixels we shift the filter to get the next output position. With stride 1 we move the filter one pixel at a time; with stride 2 we jump two pixels, skipping intermediate locations. As illustrated in panel (a) of Figure 40, increasing the stride makes the output smaller because we take fewer positions, and it also reduces computation.
- **Padding and output size.**
  If we slide a 3×3 filter over a finite image without touching the borders, the output becomes smaller than the input. To avoid losing resolution, we can use **zero-padding** (also called "same" padding): we pad the image with zeros around the edges so that the filter can still be centred near the boundary. In panel (b) of Figure 40, the outer red frame of zeros shows such padding; with appropriate padding the output feature map can be kept at the **same spatial size** as the input.

These hyperparameters let us trade off spatial resolution, number of features and computational cost. Later, when designing CNN architectures, we will combine filter size, depth, stride and padding to control how quickly the spatial dimensions shrink and how rich the learned representation becomes.

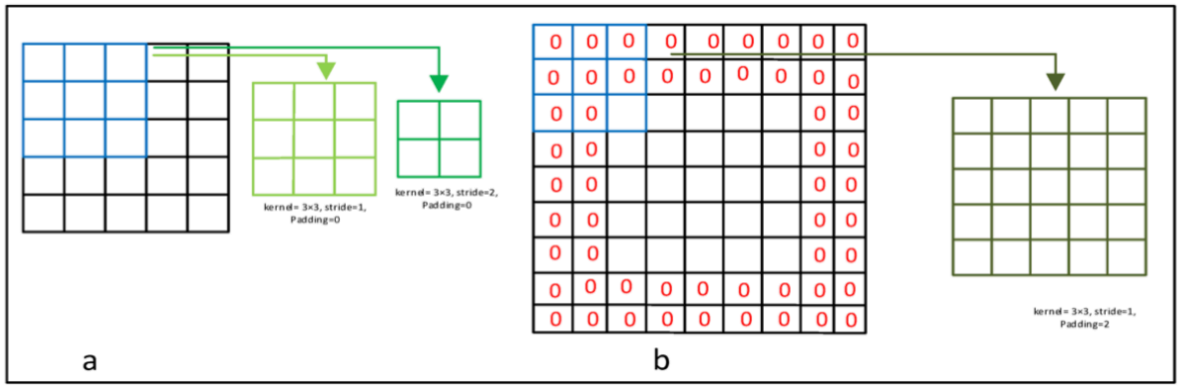

Figure 40: Convolution hyperparameters: (a) effect of stride on output size; (b) zero-padding to keep input and output sizes the same.

11.7 The detector and pooling stages

The **second stage** of the convolutional layer is the **detector stage**. Here we apply a fixed non-linearity, usually the ReLU,

$$\mathrm{ReLU}(t) = \max\{0, t\},$$

independently to every score produced by the convolution. A neuron is **active** (fires) only if the convolution score at that location is large enough, so a high ReLU output means that the feature encoded by the filter (for example a vertical edge) is present in that part of the image. This stage usually introduces **no additional parameters**, because the form of the non-linearity is chosen in advance.

The **third stage** is **pooling**, a form of **downsampling**. Pooling aggregates nearby activations into a single value, thereby reducing the spatial resolution of the feature map. The most common choice is **max-pooling**: we partition the feature map into small blocks (often 2×2) and keep only the maximum activation in each block.

In the left part of Figure 41, a 4×4 grid of activations is transformed by 2×2 max-pooling into a 2×2 grid: in each coloured 2×2 block we keep only the largest number (20, 30, 112, 37). This throws away information, but it has two important effects:

- it **reduces the image size**, so later layers have fewer units and parameters;
- it makes the representation more **robust to small translations**: if a feature (say, a vertical edge) moves slightly within a 2×2 block, the pooled value stays the same, so the network still registers that the feature is present somewhere in that region.

The right part of Figure 41 shows the same idea in a real CNN: a feature map of size $224 \times 224 \times 64$ is downsampled by pooling to $112 \times 112 \times 64$. Repeating convolution–ReLU–pooling blocks many times lets us gradually shrink the spatial dimensions while keeping only the most important activations, so that later we can afford to add dense layers on a much smaller representation.

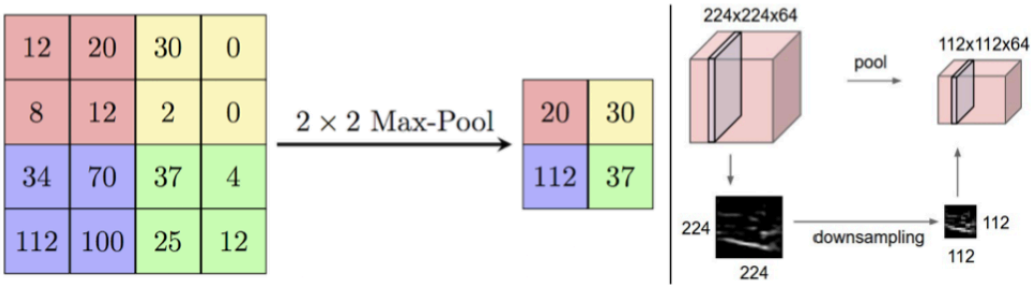

Figure 41: Detector (ReLU) and 2×2 max-pooling: local downsampling of activation maps and reduction from 224×224×64 to 112×112×64.

11.8 The convolutional layer

We can now put the three stages together. In this course, a **convolutional layer** means:

convolution operation → ReLU (detector stage) → pooling.

Sometimes in the literature only the convolution operation (possibly followed by ReLU) is called a "convolutional layer", and pooling is treated as a separate layer. Here we group them, because they are almost always used in this sequence.

In a deep CNN there are usually **several convolutional layers in a row**. Often we apply multiple convolution + ReLU blocks before doing a pooling step, then repeat this pattern at the next spatial scale. Each time we:

- use convolution + ReLU to detect more complex local features;
- use pooling to downsample and keep only the strongest activations.

As illustrated in Figure 42, the original car image is successively transformed by many CONV–ReLU–POOL stages. Early layers respond to simple patterns such as edges and corners; deeper layers respond to larger shapes and object parts. Finally, the last feature maps are fed into one or more **fully connected (FC)** layers that output class scores such as "car", "truck", "airplane", "ship", "horse".

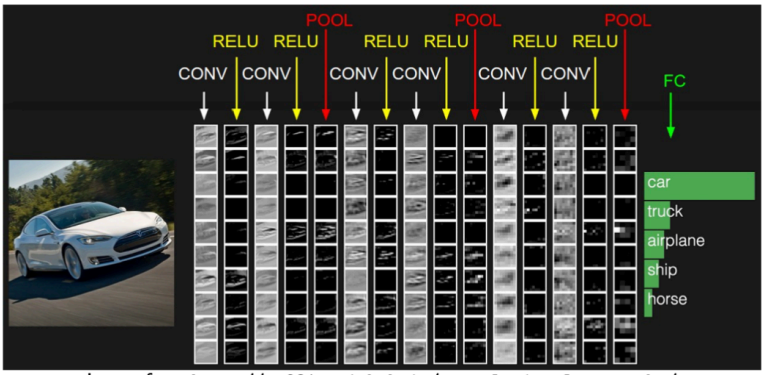

Figure 42: Several convolutional (CONV + ReLU + POOL) layers stacked before a final fully connected classifier.

11.9 Convolutional layers

A single convolutional layer usually applies **several filters in parallel** to the same input image. Each filter looks at the same local neighbourhoods (receptive fields), but with **different weights**, so it responds to different types of local patterns.

In Figure 43 we start from a greyscale image of a car. Below, Figure 44 shows four simple 4×4 filters whose entries are -1 (black) or $+1$ (white). They are chosen by hand to behave like oriented **edge detectors**:

- Filter 1 reacts to vertical transitions from dark to light (left edges).
- Filter 2 reacts to vertical transitions from light to dark (right edges).
- Filter 3 reacts to horizontal transitions from dark to light (bottom edges).
- Filter 4 reacts to horizontal transitions from light to dark (top edges).

Each filter is convolved with the car image, sliding across all positions. This produces four **activation maps**, shown in Figure 45. Bright areas in an activation map indicate locations where the corresponding pattern is strongly present. Some filters highlight vertical contours of the car, others emphasise horizontal structures such as the bonnet or bumper.

This illustrates two key ideas:

- A convolutional layer learns **multiple filters** so that different feature maps focus on different visual aspects of the same image.
- Filters are local and **shared across the whole image**, so the same pattern (for example "vertical edge") can be detected anywhere in the image.

In real CNNs, the filter values are not hand-crafted as here; they are **learned from data** via gradient descent so that the activation maps become useful for the final prediction task.

Figure 43: Greyscale input image of a car

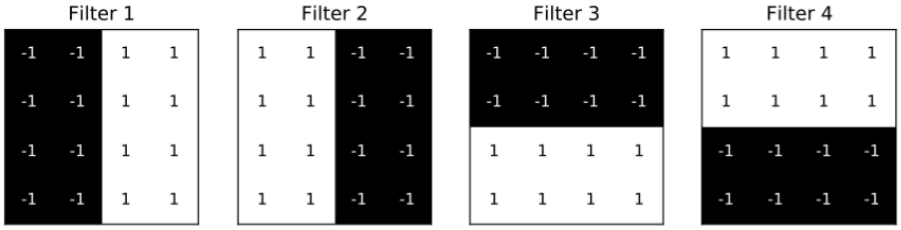

Figure 44: Four 4×4 edge-detecting filters

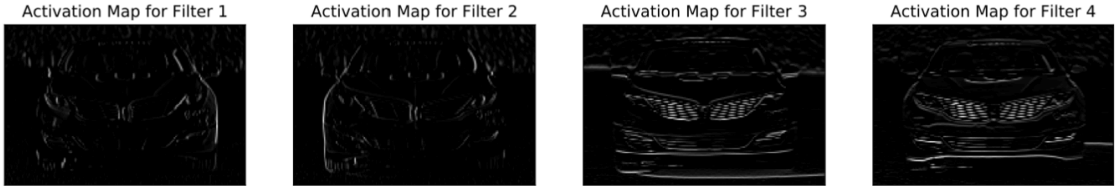

Figure 45: Activation maps for the four filters

11.10 Convolutional layer

Mathematically, a convolutional layer combines the three stages we have seen:

1. **Linear convolution stage**
2. **Detector stage (ReLU)**
3. **Pooling stage**

For one feature map, we can write these stages as

$$Z_j^{(1)} = b^{(1)} + w_j^{(1)\top} X, \quad j = 1, \dots, m_1,$$

$$Z_j^{(2)} = \max\{0, Z_j^{(1)}\}, \quad j = 1, \dots, m_1,$$

$$Z_j^{(3)} = \text{pool}(Z^{(2)})_j, \quad j = 1, \dots, m_3.$$

Here X is the vector of input pixels (possibly after zero-padding), $Z^{(1)}$ are the convolution scores, $Z^{(2)}$ the ReLU activations, and $Z^{(3)}$ the pooled outputs. The index j runs over spatial locations: each $Z_j^{(1)}$ corresponds to one receptive field of the filter.

In a **convolutional** layer, the parameters are heavily constrained:

- All biases are equal: $b_1^{(1)} = \cdots = b_{m_1}^{(1)} = b^{(1)}$.
- Each weight vector $w_j^{(1)}$ has non-zero entries only for pixels inside the receptive field of the filter; all other entries are zero.
- Moreover, the non-zero pattern is the **same for every** j: the vectors $w_1^{(1)}, \dots, w_{m_1}^{(1)}$ are just spatially shifted copies of the same filter weights, aligned with different receptive fields. This is the formal way of expressing **weight sharing**.

As illustrated in the 1D toy example in Figure 46, a single 3×1 filter slides over an input of six pixels (plus zero-padding). The corresponding convolution stage uses the same three non-zero weights and one shared bias at every location. After ReLU and pooling, only three pooled outputs remain.

A crucial consequence is that the **number of parameters does not depend on the image size or the number of output locations**. For a single filter of height H and width W we only need

$$\text{H} \times \text{W} + 1$$

parameters (H×W filter weights plus one bias). In the example of Figure 46, this is $3 \times 1 + 1 = 4$, much smaller than a dense layer that would use a different set of weights at every position.

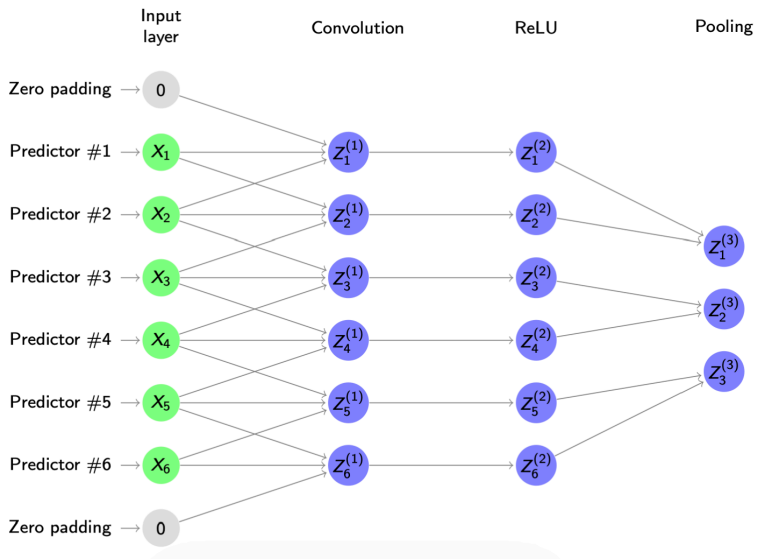

Figure 46: 1D toy example of a convolutional layer with zero-padding, convolution, ReLU and pooling, illustrating weight sharing and the small number of parameters.

11.11 A typical CNN

In practice, a convolutional layer almost never uses just one filter. Instead we apply **several filters in parallel** to the same input. The number of filters is called the **filter depth** and equals the number of output feature maps.

If each filter has height H and width W, and we use one shared bias per filter, then the total number of parameters in a convolutional layer is

$$(\mathrm{H} \times \mathrm{W} + 1) \times \text{filter depth}.$$

So the parameter cost grows with the filter size and the number of filters, but **not** with the spatial size of the input image. In the first convolutional layer of Figure 47 we use 10 filters of size 5×5, each with one bias:

$$(5 \times 5 + 1) \times 10 = 260 \text{ parameters}.$$

A dense layer connecting a 28×28 input directly to, say, 100 hidden units would already need $28 \times 28 \times 100 = 78\,400$ weights, so the saving is substantial.

The architecture in Figure 47 shows a **typical CNN for MNIST digits**:

- Input: a 28×28 greyscale image of a digit.
- First convolutional layer: 10 filters of size 5×5 produce 10 feature maps of size 24×24.
- Max-pooling layer: 2×2 pooling reduces these to $12 \times 12 \times 10$.

- Second convolutional and pooling layers: more filters extract higher-level features and further reduce the spatial size (down to $4 \times 4 \times 20$ in the sketch).

Before the final prediction, a **flattening layer** simply reshapes the last $4 \times 4 \times 20$ tensor into a 1D vector so that we can use a standard **fully connected layer** (with dropout for regularisation) to output 10 class scores, one for each digit. The convolution–pooling blocks therefore act as a trainable feature extractor, and the dense layers on top perform classification.

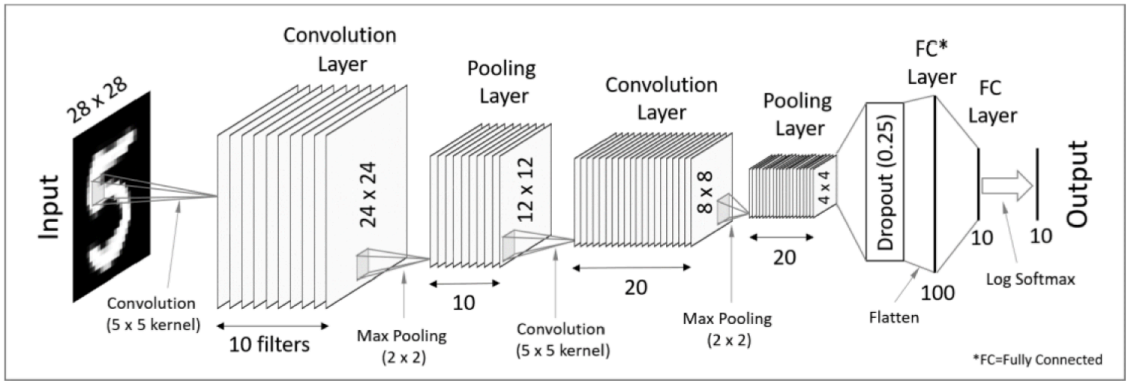

Figure 47: Example of a typical CNN architecture for MNIST digits, with two convolution–pooling blocks followed by flattening and fully connected layers.

11.12 Digit classification with neural networks

We return to the handwritten digit data set and compare a range of models, from classical linear methods to deep neural networks. All methods are trained on the same training set and evaluated on an independent test set. We consider both a **fully connected neural network (FC NN)** and a **convolutional neural network (CNN)**.

The table below shows that several methods (RF, FC NN, CNN) achieve essentially **zero training error**, meaning they fit the training digits almost perfectly. However, their **test errors** are quite different: the CNN reaches about 3.2% test error, clearly better than all other methods, including the FC NN (about 5.8%) and strong baselines such as the radial SVM (about 5.4%). This illustrates two points stressed in the lecture:

- Training error alone is not informative once models are sufficiently flexible; we must compare **generalisation performance** on unseen data.
- CNNs gain a substantial advantage by exploiting the **spatial structure** of images, whereas fully connected networks and other methods treat pixels as unordered features.

Models compared:

- **linreg:** direct linear regression.
- **LDA1:** Linear Discriminant Analysis (LDA) based on the values x_i.
- **LDA2:** LDA based on x_i and x_i^2.
- **QDA:** Quadratic Discriminant Analysis with a distinct covariance matrix for each class.
- **log:** logistic regression (multinomial).
- **log-ridge:** logistic regression with ridge penalty.
- **log-lasso:** logistic regression with lasso penalty.
- **RF:** random forest.
- **linear SVM:** linear support vector classifier.
- **radial SVM:** SVM with radial basis kernel.
- **FC NN:** fully connected neural network.
- **CNN:** convolutional neural network.

Training and test errors (%)

Method	Training	Test
linreg	7.6	13.1
LDA1	6.2	11.5
LDA2	3.9	10.2
QDA	1.8	13.5
log	0.01	11.1
log-ridge	4.1	9.0
log-lasso	2.7	8.8
RF	0.0	5.9
linear SVM	1.5	6.5
radial SVM	0.6	5.4
FC NN	≈ 0.0	5.8
CNN	≈ 0.0	3.2

11.13 Activations and filters of the first layer

The first convolutional layer in a CNN is not yet trying to decide whether an image is a 2, 4, 6, etc. Its role is to extract very **generic visual features** that will be reused by deeper layers.

In Figure 48, each row shows:

- on the left, an input digit image (2, 4, 9, 6, …);
- next to it, the corresponding **activation maps** produced by several learned filters in the first layer;
- at the bottom, some of the **filters themselves**, visualised as small greyscale patches.

These learned filters behave like simple edge or stroke detectors:

- some respond strongly to **horizontal transitions** from white to black;
- others to **vertical strokes**;
- others to slightly **diagonal curves** or to regions with a lot of ink (thick strokes).

Where a particular pattern appears in the digit (for example, the horizontal top of a 2 or the vertical spine of a 4), the matching filter produces a **high activation** in that region of its activation map. Thus, each filter specialises in one basic pattern, and its activation map shows *where* that pattern is present in the image.

Two important points the script emphasises:

- In the **first layer**, filters and activations are still closely related to the raw pixels, so we can interpret them visually.
- Deeper layers then **combine** these basic features (edges, blobs, simple shapes) into more complex patterns that are harder to interpret directly but more useful for classification.

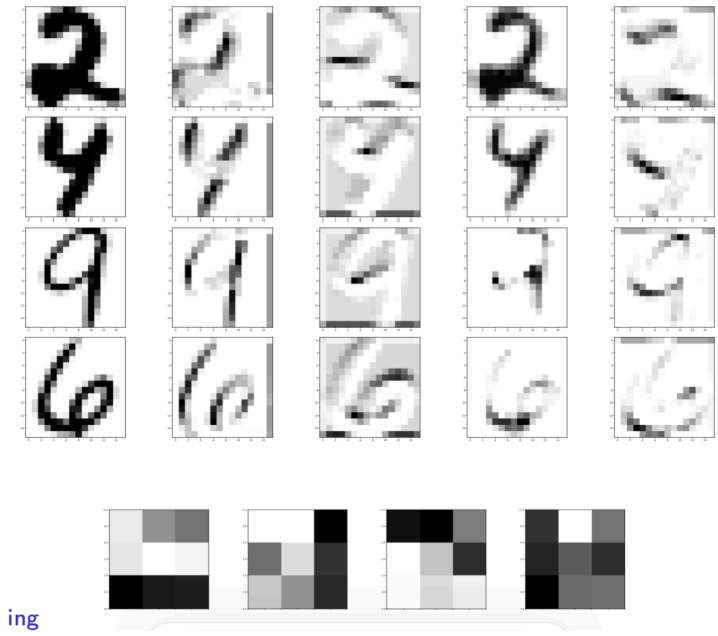

Figure 48: Activations and filters of the first convolutional layer: each row shows an input digit, the corresponding activation maps of several filters, and examples of the learned filters themselves.

12 Deep learning: Further topics

12.1 ImageNet

To move beyond relatively small benchmarks such as CIFAR-10, computer vision models are often evaluated on **ImageNet**, a much more complex data set that underpinned the ImageNet Large Scale Visual Recognition Challenge (ILSVRC).

Compared with CIFAR-10, ImageNet is harder in several ways:

- It contains around **200 object categories**, rather than just 10.
- **Multiple objects can appear in one image**, so a single picture may be labelled with several classes (e.g. bird and frog, or person, dog and chair).
- The data set is **much larger**, with roughly **450 000 training images** and **40 000 test images**.
- Images have **higher and variable resolution**; the average size is about 482×415 pixels, so each raw image is on the order of $2 \cdot 10^5$ pixels instead of 1024.

Because of this combination of many classes, multiple objects, and high-resolution images, ImageNet became the standard benchmark for testing new convolutional network architectures (AlexNet, VGG, ResNet, and others). In the next sections we will see how such large ImageNet-trained CNNs can be reused for new tasks through **transfer learning**.

12.2 Popular network architectures

Convolutional neural networks were first developed in the 1990s, notably by Yann LeCun and co-authors. The best-known early architecture is **LeNet**, designed to read handwritten zip codes (essentially the MNIST digit data). LeNet already used the basic building blocks we have seen:

- a first **convolutional layer**,
- a **subsampling / pooling** layer,
- another convolution + pooling block,
- followed by one or two **fully connected layers** for classification.

This architecture is relatively shallow by today's standards, but it already illustrates two key ideas:

- convolution + pooling layers act as an **automatic feature extractor**, turning raw pixels into increasingly abstract features;
- as we go deeper, **learned features become more complex**: early filters detect simple strokes or edges, later ones combine these into shapes that resemble parts of digits.

Since LeNet, a whole **"zoo" of CNN architectures** has been proposed, each trying to improve benchmark performance on data sets such as CIFAR-10 and ImageNet. Examples include AlexNet, VGG, Inception and ResNet. They mainly differ in how many layers they use, how they arrange convolution

and pooling, and how they choose filter sizes, but they all build on the same core principles introduced by LeNet.

12.3 AlexNet

AlexNet (Krizhevsky, Sutskever and Hinton, 2012) was the network that truly popularised CNNs in modern computer vision. It introduced several important changes compared with earlier architectures like LeNet:

- It used **ReLU activations** systematically instead of sigmoids, which made optimisation easier and allowed much deeper networks to be trained.
- It was **deeper and wider**: many more filters per layer and more convolutional layers stacked on top of each other.
- It often applied **several convolution + ReLU layers before pooling**, rather than always pooling immediately after each convolution.

Trained on the ImageNet challenge, AlexNet reduced the classification error from about **26 % to 16 %**, a dramatic improvement at the time. Conceptually, however, its structure is still similar to LeNet:

- a stack of convolutional layers (with ReLU and occasional max-pooling) that gradually extract higher-level visual features;
- followed by a few large **fully connected layers** that do the final classification.

The main step forward was not a completely new idea, but the ability to **train a much deeper, higher-capacity CNN** successfully, showing that such networks can scale to large, high-resolution image data sets like ImageNet.

12.4 VGGNet

The winner of the ILSVRC 2014 ImageNet challenge was **VGGNet** (Simonyan and Zisserman). Its main message was that the **depth of the network is critical** for good performance: by simply stacking many small convolutional layers, one can dramatically improve accuracy.

VGGNet has a very **homogeneous architecture**: from input to the last convolutional block it uses only

- 3×3 convolutions, and
- 2×2 max-pooling,

repeated many times. A typical configuration has **16 weight layers** (convolution + fully connected). The downside is that this simplicity comes at a cost: VGGNet has on the order of **140 million parameters**, most of them in the fully connected layers, so it requires a lot of memory and computation.

12.5 More networks

After AlexNet and VGGNet, many other CNN architectures were proposed, often motivated by specific applications or by the desire to go deeper without losing trainability. Examples include:

- **ResNet**, which introduces *skip connections* so that very deep networks (50+ layers) can be trained reliably.
- **Inception** (GoogleNet), which mixes different filter sizes in parallel within the same layer to capture information at multiple spatial scales.

There is now a large variety of architectures – convolutional, recurrent, residual, generative, etc. – and new variants keep appearing as research in deep learning for vision and other domains remains very active.

12.6 Typical filter weights of the first layer

The **filters of the first convolutional layer** can often be visualised directly, since they operate on the raw pixels. As shown in Figure 49, when we plot these filters for a CNN trained on natural images, they typically look like:

- oriented **edge detectors** (similar to Gabor filters), sensitive to lines at different angles;
- small **blob or colour detectors**, sensitive to local spots of particular colours.

These are very **generic features**: they do not "know" about specific objects such as cats or cars, but they capture basic visual primitives (edges, corners, simple textures) that are useful for almost any vision task. Deeper layers then build on these primitives to form more complex and task-specific features, which are harder to interpret visually.

12.7 Unconverged filters

On the previous slide we saw clean first-layer filters that behaved like edge or blob detectors. In contrast, as shown in Figure 50, sometimes the learned filters look more like noisy static: they do **not** show clear structure and do not extract clear features.

There are several possible reasons for such **unconverged filters**:

- The network has **not been trained long enough**, so optimisation stopped before reaching a good minimum.
- The **learning rate** is badly chosen (for example too high), so stochastic gradient descent keeps bouncing around instead of settling.
- **Weight regularisation** is too weak, which allows very jagged, noisy filters instead of smooth, interpretable ones.

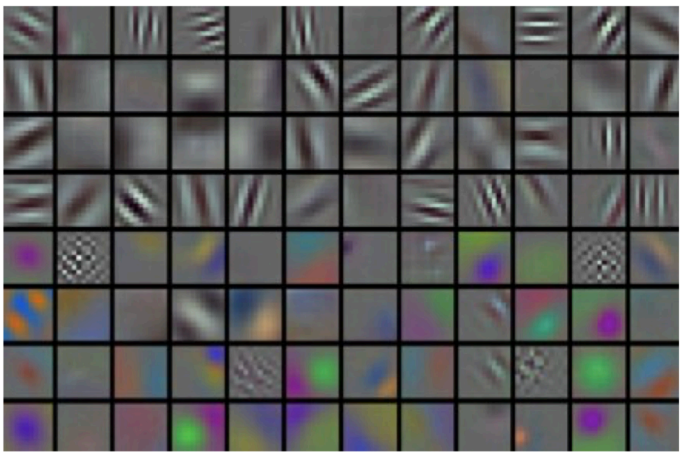

Image from http://cs231n.github.io/neural-networks-3/

Figure 49: Typical first-layer filters of a CNN trained on natural images. Many resemble oriented edge detectors or small colour blobs.

When you visualise first-layer filters and they all look like noise, it is a warning sign that the training setup (epochs, learning rate, regularisation) needs to be revisited.

12.8 Activations of deeper layers

For early layers we can interpret the filters themselves (edges, colours, textures). For deeper layers this is much harder, because each filter combines many earlier features. Instead, we look at **which input images maximally activate a given neuron**.

A common procedure is:

1. Pick a neuron in a **deep layer** (here: the 5th and last pooling layer of AlexNet).
2. Run a large collection of images through the network.
3. For that neuron, select the images where its activation is **highest**.
4. Display those images in one row.

As shown in Figure 51, each row in the visualisation corresponds to one deep neuron. We see, for example:

- a neuron that fires mostly on **human faces**;
- one that responds strongly to **dogs** and similar animal faces;
- one that prefers **text or number patterns**;
- others that specialise in **houses** or bright **round highlights**.

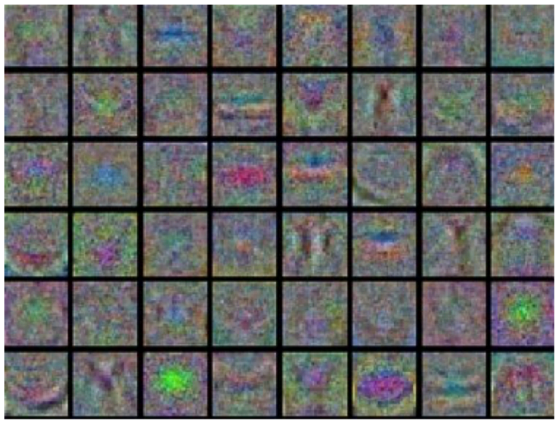

Figure 50: Example of unconverged first-layer filters that look noisy and do not exhibit clear edge or blob patterns.

The neuron does not "know" the semantic label; it simply responds to a particular **visual pattern** that tends to appear in those objects. This illustrates the general trend:

- **Early layers** learn generic, low-level features (edges, colours, textures).
- **Deeper layers** learn features that are **specific to the data set and task**, and begin to resemble human-interpretable concepts such as faces, dogs or buildings.

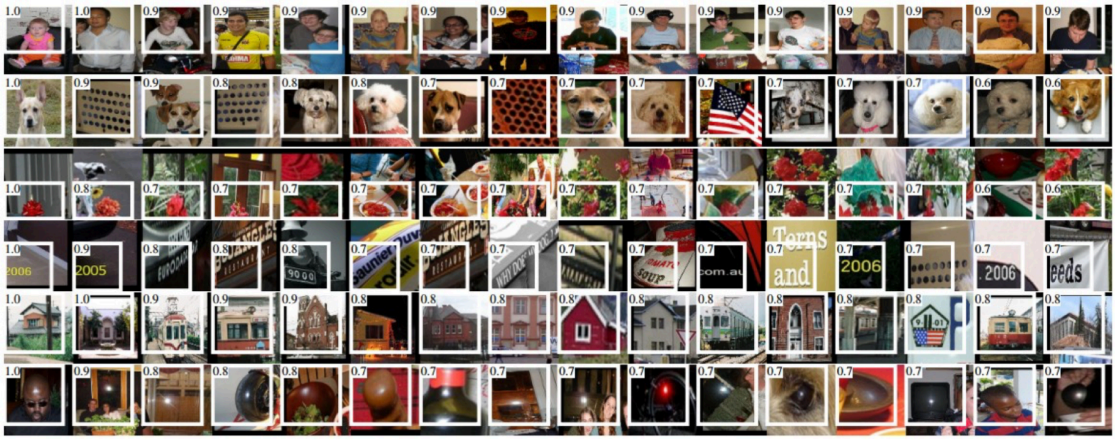

Image from Girshick et al. (2013, https://arxiv.org/abs/1311.2524)

Figure 51: Images that maximally activate six neurons in a deep pooling layer of AlexNet, showing specialisation to faces, dogs, text, houses and other high-level patterns.

12.9 Pre-trained networks

Modern CNNs (AlexNet, VGG, ResNet, …) have tens or hundreds of millions of parameters and were trained on huge data sets such as ImageNet. For these models, the **expensive part is fitting** (backpropagation and optimisation), not the forward pass.

As an end-user you may not have:

- enough **data** to train such a network from scratch; or
- enough **computational resources** to run the full training.

A practical alternative is to use a **pre-trained network**:

- Take a state-of-the-art CNN that has already been trained on a large benchmark like ImageNet.
- Download its **weights** from a public "model zoo".
- Use it directly to perform **exactly the same task** it was trained on (same input size, same output label set).

This setup lets you benefit from a powerful feature extractor without paying the training cost yourself. However, it only solves the *original* task (e.g. ImageNet classification). To adapt the network to a **new task or new classes**, we need **transfer learning**, where we keep most of the pre-trained network and retrain only the final layers (or fine-tune a few deeper layers) on our own data set.

12.10 Transfer learning

A main limitation of a **pre-trained network** is that it can only predict the **pre-defined categories** it was originally trained on (for example the 1000 ImageNet classes). In many practical applications, however, you care about **different categories**, such as:

- flower species,
- cancer types,
- car brands,
- traffic signs,
- or even astronomical objects such as different types of galaxies.

At the same time, you often have only a **small number of labelled images** for your own problem – far too few to train a large CNN from scratch.

Transfer learning addresses this situation. The idea is to:

1. start from a CNN that has been **pre-trained on a large data set** such as ImageNet;
2. **reuse what it has already learned** about generic visual features;
3. adapt only part of the model to your specific task.

For example, you might use an ImageNet-trained network as the starting point for a classifier that distinguishes between several kinds of galaxies.

12.11 Transfer learning in practice

How does this work in practice? As shown in Figure 52, it helps to view a CNN as consisting of two conceptual parts:

1. a **generic feature extractor** – all the convolution + pooling layers;
2. a **problem-specific classifier** – the last fully connected layers that map features to output classes.

In transfer learning we typically:

- keep the **feature extractor** from the pre-trained network (possibly freezing its weights, or fine-tuning only slightly), because it already encodes useful edges, textures and shapes;
- **replace the final classifier** with a new one adapted to our labels (for example a new softmax layer with as many outputs as our target classes), and train only this part on our smaller data set.

This way we benefit from the rich, generic visual representation learned from millions of ImageNet images, while only needing to learn a relatively small number of additional parameters for the new classification task.

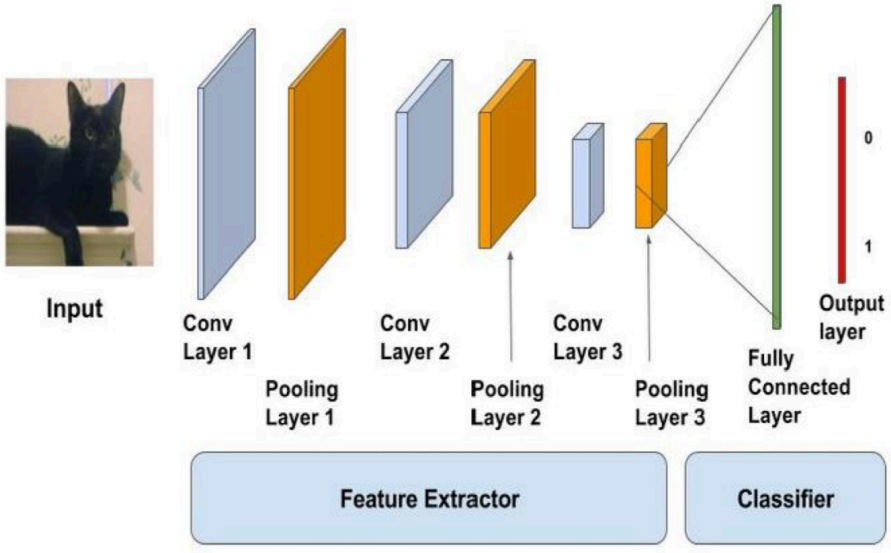

Figure 52: Transfer learning: reuse the convolutional feature extractor and retrain a new task-specific classifier on top.

12.12 Transfer learning: mathematical view

We can describe transfer learning more formally by splitting a CNN into two functions:

- a **feature extractor** that maps an input image x to a vector of feature activations

$$z = f(x) = (z_1, \dots, z_m),$$

- a **classifier** that maps the features to output scores or probabilities

$$y = g(z).$$

When a network is trained on a large data set such as ImageNet, we learn parameters θ_f of the feature extractor and θ_g of the classifier:

$$z = f_{\theta_f}(x), \qquad y = g_{\theta_g}(z).$$

In **transfer learning** for a new, smaller data set we proceed as follows:

1. **Start from the ImageNet-trained network.**
   The convolution + pooling layers define f_{θ_f}; the final fully connected layers define g_{θ_g}.
2. **Freeze the feature extractor.**
   We keep θ_f fixed and reuse it as a generic map that turns any input image into a feature vector $z = f_{\theta_f}(x)$.
3. **Replace and retrain the classifier.**
   We discard the original last layer(s), introduce a new classifier g_ϕ with parameters ϕ (for example a softmax over our own classes), and train only ϕ on the small task-specific data set:

$$y = g_\phi(f_{\theta_f}(x)).$$

In the left panels of the slide (labelled "Train on ImageNet" and "Small data set feature extractor"), this corresponds to:

- keeping all convolutional blocks and the first fully connected layers as the **frozen feature extractor**;
- retraining only the final fully connected layers on the new labels.

Mathematically, transfer learning is thus the reuse of a pre-trained feature map f_{θ_f} and the **replacement of the last part of the network** by a new classifier tailored to the target problem.

12.13 Transfer learning: practical strategy

A simple and widely used transfer-learning strategy is illustrated in Figure 53:

1. **Start from a pre-trained CNN**, typically trained on ImageNet (for example **AlexNet**).
2. **Remove the last fully connected layer**, whose output are the 1000 ImageNet class probabilities.

3. **Freeze all remaining layers** (convolution, pooling and the first fully connected layers) and use them as a **fixed feature extractor**.
4. For each new image, pass it through the network and take the activations of the **last but one layer** (sometimes called the *code*).
    - For AlexNet this code is a vector in $\mathbb{R}^{4096}$.
5. **Train a separate linear classifier on these codes**, for instance:
    - multinomial logistic regression, or
    - a linear SVM.

In this way, the heavy convolutional part of the network is reused without change, and only a light-weight linear model is fitted on top of the learned representation for the new task.

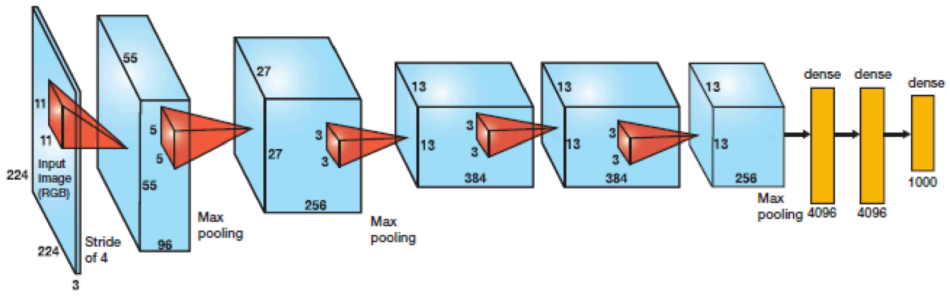

Figure 53: Transfer-learning strategy: use AlexNet (trained on ImageNet) as a fixed feature extractor and train a new linear classifier on the 4096-dimensional codes from the last but one layer.

12.14 Transfer learning: fine-tuning strategy

An alternative to using a pre-trained network only as a fixed feature extractor is to **fine-tune** (parts of) the CNN.

Instead of freezing all convolutional layers and training only a new classifier, we can:

- keep the **early layers fixed**, because they learn very generic features such as edge or blob detectors that are useful for many tasks;
- **unfreeze some of the later layers** (often the last few convolutional blocks and/or fully connected layers) and continue training them on the new data set together with the new classifier.

This makes sense because:

- early layers mainly capture low-level structure that transfers well between data sets;
- **later layers are more specific** to the original training data, so adapting them helps the network specialise to the new task.

Fine-tuning therefore lies between two extremes: - at one end, we train only a new linear classifier on fixed features; - at the other end, we train the whole network from scratch. It allows us to reuse most of the pre-trained representation while still adapting higher-level features to the new problem.

12.15 Choosing a transfer-learning strategy

Which transfer-learning strategy to use depends on several factors, in particular:

- the **size** of your data set;
- the **similarity** between your data and the data used to train the original network.

A useful rule of thumb is:

- If your data set is **small and very similar** to ImageNet (or the original data), you should generally **train only the final classifier** and keep the rest of the network fixed, to avoid overfitting.
- If your data set is **large** and **not too different** from the original data, you can additionally **fine-tune some of the deeper layers** (often with a smaller learning rate) with relatively low risk of overfitting.

In practice, many applications start with the simple "fixed feature extractor + new classifier" set-up, and then experiment with gradually unfreezing deeper layers if enough data are available.

12.16 Further topics

Classical deep neural networks are only the starting point. Around them there is a whole ecosystem of architectures and applications that keeps growing.

- **Recurrent neural networks (RNNs).**
  Designed for **sequences** rather than static inputs. They are widely used for time-series forecasting, natural language, speech, and any setting where past inputs influence future predictions. The key idea is that the network maintains a **hidden state** that propagates through time and summarises past information.
- **Generative Adversarial Networks (GANs).**
  GANs are **generative models**: instead of predicting labels, they generate new samples that look like the training data. A *generator* network tries to create realistic images, while a *discriminator* network tries to distinguish real from fake. Training them together leads to very sharp, realistic samples and underlies technologies such as deepfakes.
- **Autoencoders (and variational autoencoders).**
  Autoencoders learn to **compress data into a low-dimensional code** and then reconstruct it. They are used for dimension reduction, denoising and representation learning. Variational autoencoders add a probabilistic layer and can also be used as generative models.

- **Deep reinforcement learning.**
  In reinforcement learning, an agent must learn how to act in an environment to maximise long-term reward. Deep networks are used to approximate value functions or policies when states are high-dimensional (e.g. images), enabling applications such as game-playing agents and robotics.

Overall, deep learning is not a single model but a **family of tools**: convolutional networks for images, RNNs for sequences, GANs and autoencoders for generation and compression, and deep RL for sequential decision-making.

12.17 Reinforcement learning

Reinforcement learning formalises **learning by trial and error**.

- There is an **agent** (for example a robot, a trading algorithm, or a game-playing program) that repeatedly interacts with an **environment**.
- At each time step the agent observes a **state**, chooses an **action**, and receives a **reward** plus a new state.
- Over many iterations the agent aims to learn a **strategy of action** (a *policy*) that maximises the expected **long-term cumulative reward**, not just the immediate one.

In simple settings the optimal policy can sometimes be computed exactly. In more realistic settings the state is high-dimensional (e.g. raw images of a game screen) and the dynamics are complex. Here **deep neural networks** are used inside the RL algorithm to approximate:

- a **value function** (how good a state or state–action pair is), or
- a **policy** (a mapping from states to action probabilities).

This combination, known as **deep reinforcement learning**, has enabled systems that learn to play Atari games directly from pixels and programmes such as AlphaGo, which learned to play Go at superhuman level by combining deep networks with reinforcement learning.